

Chapter 3: Generating Random Variables

Lectures on Monte Carlo Theory — Leskelä & Vihola

- 1 3.1 Inverse Transform Method (ITM)
- 2 3.2 ITM Variants for Lattice Distributions
- 3 3.3 Ad Hoc Methods
- 4 3.4 Uniform Distribution on Geometric Shapes
- 5 3.5 Acceptance-Rejection Method
- 6 3.6 Composition Method
- 7 3.7 Generating Gaussian Random Numbers
- 8 3.8 Uniform Distribution on Balls, Spheres, Ellipsoids
- 9 3.9 Bivariate Vectors: Conditional ITM and Copulas
- 10 3.10 Heaviness of Distributions
- 11 Summary

Outline

- 1 3.1 Inverse Transform Method (ITM)
- 2 3.2 ITM Variants for Lattice Distributions
- 3 3.3 Ad Hoc Methods
- 4 3.4 Uniform Distribution on Geometric Shapes
- 5 3.5 Acceptance-Rejection Method
- 6 3.6 Composition Method
- 7 3.7 Generating Gaussian Random Numbers
- 8 3.8 Uniform Distribution on Balls, Spheres, Ellipsoids

3.1 The Core Problem: Sampling from Any Distribution I

The central challenge of this chapter is the following: a computer's random-number generator can only produce a stream of *uniform* random numbers $U_1, U_2, \dots$ drawn independently from $\mathcal{U}[0, 1)$, the uniform distribution on the interval $[0, 1)$. But in a simulation we typically need samples from other distributions — exponential waiting times, normal measurement errors, Pareto-distributed losses, and so on. How do we convert uniform random numbers into samples from an arbitrary target distribution?

The *Inverse Transform Method* (ITM) gives the cleanest possible answer: if we know the inverse of the target distribution's cumulative distribution function (c.d.f.), we can transform a single uniform draw directly into a draw from the target. Throughout this chapter, $U, U_1, U_2, \dots$ always denote independent $\mathcal{U}[0, 1)$ random variables without further explanation.

3.1 The Core Problem: Sampling from Any Distribution II

Background: Cumulative Distribution Function

For a random variable X , the **c.d.f.** is the function

$$F(x) = \mathbb{P}(X \leq x), \quad x \in \mathbb{R}.$$

Here $\mathbb{P}$ (blackboard-bold P) denotes probability. The c.d.f. F is non-decreasing, right-continuous, and satisfies $F(-\infty) = 0$, $F(+\infty) = 1$. Knowing F is equivalent to knowing the entire distribution of X .

3.1 The Core Problem: Sampling from Any Distribution III

To apply ITM we need to “invert” the c.d.f., which requires care when F has flat regions (where the inverse is not unique) or jumps (where some values of u are skipped). The *generalized inverse* handles both cases correctly.

Definition 3.1 — Generalized Inverse (Quantile Function)

The generalized inverse of a c.d.f. F is defined as

$$F^{\leftarrow}(u) = \inf\{x \in \mathbb{R} : u \leq F(x)\}, \quad u \in (0, 1).$$

The notation $\inf$ (infimum) means “the smallest value of x such that $F(x)$ is at least u .” When F is strictly increasing and continuous, this is simply the ordinary inverse function F^{-1} . In general, $F^{\leftarrow}$ is non-decreasing and left-continuous.

3.1 The Core Problem: Sampling from Any Distribution IV

Notation for Definition 3.1

- $u \in (0, 1)$: a probability value (a number between 0 and 1 exclusive), also called a *quantile level*.
- $F(x)$: the cumulative probability up to x (i.e., the probability that X falls at or below x).
- $\inf\{\dots\}$: the greatest lower bound of a set; here it picks the leftmost x satisfying the inequality.
- $F^{\leftarrow}(u)$: the u -quantile of F ; for example $F^{\leftarrow}(0.5)$ is the median of the distribution.

The *tail function* (also called the survival function or complementary c.d.f.) is $\bar{F}(x) = 1 - F(x) = \mathbb{P}(X > x)$. Its generalized inverse satisfies $\bar{F}^{\leftarrow}(u) = F^{\leftarrow}(1 - u)$, which is sometimes more convenient for deriving formulas, as we will see in the exponential example.

3.1 The Core Problem: Sampling from Any Distribution V

The generalized inverse has six fundamental properties. Understanding them is important because they appear in proofs throughout the chapter.

Properties (P1)–(P6) of $F^{\leftarrow}$

- (P1) $F^{\leftarrow}(u)$ is non-decreasing and left-continuous in u , meaning it never decreases as u increases, and its value at a point equals the limit from the left.
- (P2) If F is strictly increasing and u is in the range of F , then $u = F(x)$ if and only if $F^{\leftarrow}(u) = x$ — the inverse is exact.
- (P3) If u is *not* in the range of F (a gap corresponding to a jump in F), then $F^{\leftarrow}(u) < x$ is possible even though $F(F^{\leftarrow}(u)) < u$.
- (P4) The composition $F^{\leftarrow}(F(x)) \leq x$ always holds; with equality when F is strictly increasing.
- (P5) The composition $F(F^{\leftarrow}(u)) \geq u$ always holds; with equality when F is continuous.
- (P6) Combining (P4) and (P5): $F^{\leftarrow}$ and F are approximate inverses of each other, but the equality $F(F^{\leftarrow}(u)) = u$ requires F to be continuous, and $F^{\leftarrow}(F(x)) = x$ requires F to be strictly increasing.

3.1 The Core Problem: Sampling from Any Distribution VI

The most practically useful consequence is the following equivalence, which is the mathematical engine behind all ITM algorithms.

Lemma 3.1.1 — Fundamental Equivalence

For any c.d.f. F and any $u \in (0, 1)$, $x \in \mathbb{R}$:

$$u \leq F(x) \iff F^{\leftarrow}(u) \leq x.$$

As a corollary, when F is continuous, $F(F^{\leftarrow}(u)) = u$.

Proof. The implication follows directly from the definition of the infimum: $F^{\leftarrow}(u) = \inf\{t : u \leq F(t)\}$, so $F^{\leftarrow}(u) \leq x$ is equivalent to saying that x belongs to the set $\{t : u \leq F(t)\}$, i.e. $u \leq F(x)$.

3.1 The Core Problem: Sampling from Any Distribution VII

How to Read Lemma 3.1.1

This says that asking “is the probability level u below the c.d.f. at x ?” is *exactly the same question* as asking “does the quantile at level u fall at or below x ?” This equivalence lets us translate probability statements about F into statements about $F^{\leftarrow}$, which is the key trick in the proof of the ITM theorem below.

3.1 The Core Problem: Sampling from Any Distribution VIII

CDF and its Generalised Inverse

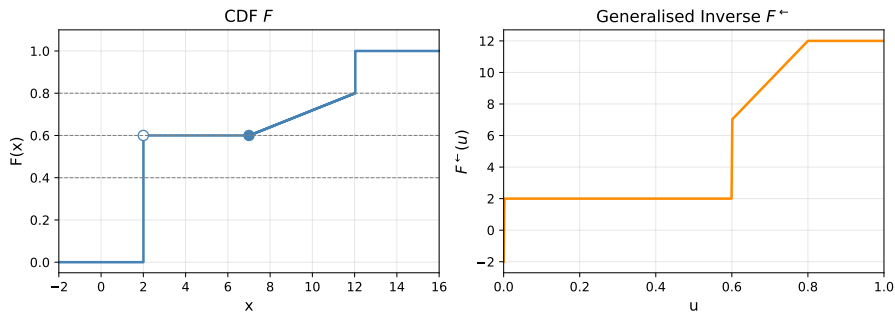


Figure: A c.d.f. F with a flat region (left figure, shown by a dashed horizontal line at level u_1) creates a jump in $F^{\leftarrow}$: several values of u all map to the same quantile. Conversely, a jump in F (a vertical step, right figure) creates a flat region in $F^{\leftarrow}$: a whole interval of quantile levels maps to the same x value. The point $x_1 = F^{\leftarrow}(u_1)$ is the leftmost x with $F(x) \geq u_1$.

3.1 The ITM Theorem and Algorithm I

We now state and prove the central result that justifies the entire method. The theorem has two parts: part (a) says we can *generate* a sample from F using a single uniform, and part (b) says the inverse is also true — applying F to a sample from F recovers a uniform.

Proposition 3.1.2 — The Inverse Transform Theorem

- Ⓐ **Sampling direction:** If $U \sim \mathcal{U}[0, 1)$, then $X = F^{\leftarrow}(U)$ has distribution F , meaning $\mathbb{P}(X \leq x) = F(x)$ for all $x \in \mathbb{R}$.
- Ⓑ **Probability-integral transform:** If F is continuous and $X \sim F$ (i.e. X has c.d.f. F), then $F(X) \sim \mathcal{U}[0, 1)$.

Proof of part (a) — step by step. We want to show $\mathbb{P}(F^{\leftarrow}(U) \leq x) = F(x)$. We apply Lemma 3.1.1, which says $F^{\leftarrow}(U) \leq x$ happens if and only if $U \leq F(x)$:

$$\mathbb{P}(F^{\leftarrow}(U) \leq x) \stackrel{\text{Lemma 3.1.1}}{=} \mathbb{P}(U \leq F(x)) \stackrel{U \sim \mathcal{U}[0,1)}{=} F(x).$$

The last step uses the fact that $\mathbb{P}(U \leq t) = t$ for any $t \in [0, 1]$ when U is uniform on $[0, 1)$.

3.1 The ITM Theorem and Algorithm II

Proof of part (b). For any $u \in (0, 1)$, by Property (P5), $F(F^{\leftarrow}(u)) = u$ when F is continuous. Therefore:

$$\mathbb{P}(F(X) \leq u) = \mathbb{P}(X \leq F^{\leftarrow}(u)) = F(F^{\leftarrow}(u)) = u.$$

Since $\mathbb{P}(F(X) \leq u) = u$ for all u , the variable $F(X)$ is $\mathcal{U}[0, 1)$.

3.1 The ITM Theorem and Algorithm III

Algorithm 6 — ITM: Generate $X \sim F$

Require: A c.d.f. F , or an efficient algorithm for computing the generalized inverse $F^{\leftarrow}(u)$ for any $u \in (0, 1)$.

Ensure: The output X has the distribution F .

- 1 Generate $U \sim \mathcal{U}[0, 1)$ (one uniform random number).
- 2 Compute $X = F^{\leftarrow}(U)$.
- 3 Return X .

A useful variant: since $1 - U \sim \mathcal{U}[0, 1)$ whenever $U \sim \mathcal{U}[0, 1)$, we can equivalently use $Y = \overline{F}^{\leftarrow}(U)$, where $\overline{F}^{\leftarrow}(u) = F^{\leftarrow}(1 - u)$ is the quantile of the tail function. This variant is convenient for distributions like the exponential where the tail function has a simpler closed-form inverse than F itself.

3.1 The ITM Theorem and Algorithm IV

Key Insight: ITM Reduces Sampling to Inversion

The ITM theorem tells us that sampling from *any* distribution reduces to exactly two steps: (1) draw one uniform number, and (2) evaluate $F^{\leftarrow}$. The entire difficulty is therefore concentrated in step (2). For many distributions, $F^{\leftarrow}$ has a simple closed form (exponential, Pareto, geometric). For the normal distribution, $F^{\leftarrow} = \Phi^{-1}$ has no closed form and must be approximated numerically.

Corollary 3.1.3 (Uniform p -values). If T is a test statistic whose c.d.f. F is continuous, then the p -value $p = 1 - F(T) = \bar{F}(T)$ is exactly $\mathcal{U}[0, 1)$ -distributed under the null hypothesis. This is the theoretical justification for the second-level uniformity testing procedure of Chapter 2.

3.1 Worked Example: Exponential Distribution $\text{Exp}(\lambda)$ I

The exponential distribution is the canonical model for waiting times between independent events — for example, the time until the next customer arrives, or the lifetime of an electronic component. Its c.d.f. is

$$F(x) = \begin{cases} 0 & x < 0, \\ 1 - e^{-\lambda x} & x \geq 0. \end{cases}$$

Notation: Exponential Distribution $\text{Exp}(\lambda)$

- λ (lambda) > 0 : the *rate parameter*, measuring how frequently events occur. A larger λ means more frequent arrivals and hence shorter average waiting times.
- $e^{-\lambda x}$: the exponential function raised to $-\lambda x$. This factor decreases from 1 (at $x = 0$) toward 0 as x grows, which is why the exponential distribution has “light tails.”
- Mean: $\mathbb{E}[X] = 1/\lambda$ (average waiting time).
- Variance: $\text{Var}(X) = 1/\lambda^2$.

3.1 Worked Example: Exponential Distribution $\text{Exp}(\lambda)$ II

To apply ITM we need $\bar{F}^{\leftarrow}$, the quantile of the tail function. Setting $\bar{F}(x) = u$, i.e. $e^{-\lambda x} = u$, and solving for x (take logarithm of both sides: $-\lambda x = \log u$, so $x = -\log(u)/\lambda$):

$$\bar{F}^{\leftarrow}(u) = -\frac{1}{\lambda} \log(u).$$

How to Read $X = -\log(U)/\lambda$

When U is close to 1 (most of the probability mass), $-\log(U)$ is close to 0, giving a small value of X — short waiting times are common. When U is close to 0 (a rare event), $-\log(U)$ is large, giving a long waiting time X — long waits are rare but possible. The rate λ simply scales the output: doubling λ halves all waiting times, keeping the same shape but compressing the distribution.

3.1 Worked Example: Exponential Distribution $\text{Exp}(\lambda)$ III

Algorithm 7 (ITM-Exp): Generate $X \sim \text{Exp}(\lambda)$

- 1 Generate $U \sim \mathcal{U}[0, 1)$.
- 2 Set $X = -\log(U)/\lambda$.
- 3 Return X .

3.1 Worked Example: Exponential Distribution $\text{Exp}(\lambda)$ IV

Worked Numerical Example

Suppose $\lambda = 2$ (so the mean waiting time is $1/\lambda = 0.5$ time units). We draw $U = 0.3$. Then:

$$X = -\frac{\log(0.3)}{2} = -\frac{-1.204}{2} = \frac{1.204}{2} \approx 0.602.$$

This particular draw gave a waiting time of about 0.602, which is above the mean of 0.5 — but not unreasonably so. Over many draws, the sample average will converge to $1/\lambda = 0.5$.

Another draw, $U = 0.9$, gives $X = -\log(0.9)/2 \approx 0.053$, a short wait. A rare draw $U = 0.01$ gives $X = -\log(0.01)/2 = 4.605/2 \approx 2.30$, a very long wait.

3.1 Worked Example: Exponential Distribution $\text{Exp}(\lambda)$ V

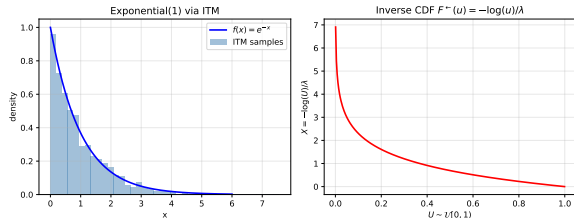


Figure: Histogram of 10 000 samples from $\text{Exp}(2)$ generated via ITM (Algorithm 7). The smooth curve is the true probability density $f(x) = 2e^{-2x}$. The histogram closely tracks the theory.

Algorithm: Inverse Transform Method (ITM)

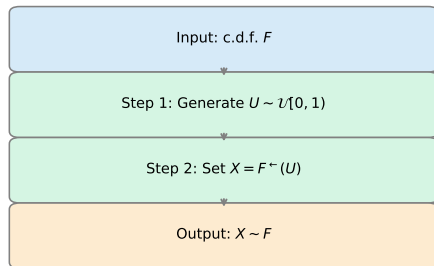


Figure: Geometric picture of ITM: a uniform draw U on the vertical axis (probability axis) is mapped horizontally to $X = F^{-1}(U)$ on the horizontal axis (value axis). Regions where F is steep (high density) produce many samples; flat regions (low density) produce few.

3.1 Worked Example: Pareto Distribution $\text{Par}(\alpha)$ I

The Pareto distribution is used to model phenomena where extreme values occur with non-negligible probability. Famous examples include the distribution of wealth (a small fraction of people hold most of the wealth), the size of insurance claims, and internet file sizes. Its c.d.f. is

$$F(x) = \begin{cases} 0 & x < 0, \\ 1 - \frac{1}{(1+x)^\alpha} & x \geq 0. \end{cases}$$

Notation: Pareto Distribution $\text{Par}(\alpha)$

- α (alpha) > 0 : the *shape parameter* (also called the *tail index*). A *smaller* α means a *heavier* tail — extreme values are more likely.
- $(1+x)^{-\alpha}$: this power-law decay is much slower than the exponential decay $e^{-\lambda x}$ of the exponential distribution, which is precisely why the Pareto has heavy tails.
- Mean: $\mathbb{E}[X] = 1/(\alpha - 1)$ for $\alpha > 1$; the mean does *not exist* (is infinite) for $\alpha \leq 1$.
- Variance: finite only for $\alpha > 2$.

3.1 Worked Example: Pareto Distribution $\text{Par}(\alpha)$ II

To apply ITM, set $\bar{F}(x) = u$, i.e. $(1+x)^{-\alpha} = u$. Raise both sides to the power $-1/\alpha$: $(1+x) = u^{-1/\alpha}$, so $x = u^{-1/\alpha} - 1$. Therefore $\bar{F}^{\leftarrow}(u) = u^{-1/\alpha} - 1$.

Algorithm 8 (ITM-Par): Generate $X \sim \text{Par}(\alpha)$

- 1 Generate $U \sim \mathcal{U}[0, 1)$.
- 2 Set $X = U^{-1/\alpha} - 1$.
- 3 Return X .

3.1 Worked Example: Pareto Distribution $\text{Par}(\alpha)$ III

Worked Numerical Example

With $\alpha = 2$ (variance is infinite for $\alpha \leq 2$, finite for $\alpha > 2$; this is the boundary case). Draw $U = 0.5$:

$$X = (0.5)^{-1/2} - 1 = \sqrt{2} - 1 \approx 0.414.$$

Draw $U = 0.01$ (a rare small uniform value, representing a tail event):

$$X = (0.01)^{-1/2} - 1 = 10 - 1 = 9.$$

Notice how a small U (probability 0.01) maps to a large $X = 9$, illustrating the heavy right tail.

Two-parameter Pareto $\text{Par}(\alpha, c)$. To create a family with a specified mean m and tail index α (for $\alpha > 1$), set $c = (\alpha - 1)m$. The c.d.f. is $F(x) = 1 - c^\alpha / (c + x)^\alpha$ for $x \geq 0$.

3.1 Worked Example: Pareto Distribution $\text{Par}(\alpha)$ IV

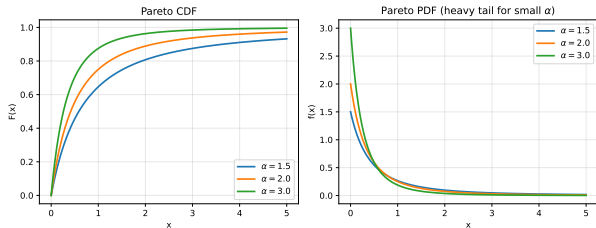


Figure: Pareto c.d.f. (top) and p.d.f. (bottom) for $\alpha = 1.5, 2$, and 3 . A smaller α (alpha) produces a heavier tail: the distribution assigns higher probability to very large values of X . For $\alpha = 1.5$, the variance is infinite.

Key Insight: Light vs. Heavy Tails

For $\text{Exp}(\lambda)$: all moments (mean, variance, etc.) are finite. Running averages of many samples converge smoothly to the true mean.

For $\text{Par}(\alpha)$: the k -th moment is finite only if $\alpha > k$. When $\alpha \leq 2$, the variance is *infinite*. Running averages converge extremely slowly, and occasional enormous values cause dramatic spikes in the sample mean.

Remarkably, the same ITM formula $X = U^{-1/\alpha} - 1$ works for any $\alpha > 0$, regardless of whether any moments are finite.

Outline

- 1 3.1 Inverse Transform Method (ITM)
- 2 3.2 ITM Variants for Lattice Distributions
- 3 3.3 Ad Hoc Methods
- 4 3.4 Uniform Distribution on Geometric Shapes
- 5 3.5 Acceptance-Rejection Method
- 6 3.6 Composition Method
- 7 3.7 Generating Gaussian Random Numbers
- 8 3.8 Uniform Distribution on Balls, Spheres, Ellipsoids

3.2 ITM for Discrete (Lattice) Distributions I

Many quantities in practice take only integer values: the number of customers in a queue, the number of defective items in a batch, the number of claims filed in a day. These are called *lattice distributions*, where the random variable X takes values in $\{0, 1, 2, \dots\}$ with probabilities $p_k = \mathbb{P}(X = k)$ for $k \geq 0$. The function $(p_k)_{k \geq 0}$ is called the *probability mass function* (PMF).

The c.d.f. of a lattice distribution is a *staircase function*: $F(x) = \sum_{k=0}^{\lfloor x \rfloor} p_k$, which jumps by p_k at each integer k and is constant between integers. Its generalized inverse is: $F^{\leftarrow}(u) = \min\{k : u \leq F(k)\}$, the smallest integer k such that the cumulative probability $F(k)$ is at least u .

Proposition 3.2.1 — Discrete ITM

Let $U \sim \mathcal{U}[0, 1)$. The random variable

$$Y = \min \left\{ k \geq 0 : U \leq \sum_{i=0}^k p_i \right\}$$

has probability mass function (p_k) .

3.2 ITM for Discrete (Lattice) Distributions II

Notation for Proposition 3.2.1

- $\sum_{i=0}^k p_i$ (Sigma, meaning “sum”): the sum of probabilities $p_0 + p_1 + \dots + p_k$, i.e. the cumulative probability $F(k)$.
- $\min\{\dots\}$: the smallest element of the set, here the smallest non-negative integer satisfying the condition.
- The algorithm scans $k = 0, 1, 2, \dots$ and stops at the first k for which the running total of probabilities reaches or exceeds U .

Proof. The event $\{Y = l\}$ occurs exactly when the cumulative probability first crosses U at step l , i.e. $F(l-1) < U \leq F(l)$. This interval has length $F(l) - F(l-1) = p_l$. Since $U \sim \mathcal{U}[0, 1)$, the probability of U falling in any interval of length p_l is exactly p_l .

3.2 ITM for Discrete (Lattice) Distributions III

Algorithm 9 (ITM-d): Generate $Y \sim (p_k)$

Require: The PMF $(p_k)_{k \geq 0}$ (all individual probabilities).

Ensure: Y follows the PMF (p_k) .

- 1 Set $Y \leftarrow 0$; generate $U \sim \mathcal{U}[0, 1)$; set $S \leftarrow p_0$.
- 2 **While** $S < U$: increment $Y \leftarrow Y + 1$, add $S \leftarrow S + p_Y$.
- 3 Return Y .

The variable S accumulates the running sum of probabilities, and we stop when the sum first reaches or exceeds U . The expected number of loop iterations equals $\mathbb{E}[Y] + 1$, because on average we step through as many values as the expected output. This means Algorithm 9 is slow for distributions with large expected values (large $\mathbb{E}[Y]$).

Efficient recursion-based variant: Algorithm 10 (ITR). When the ratio $c_{k+1} = p_{k+1}/p_k$ has a simple closed form, we can avoid storing the entire PMF and instead compute successive probabilities on the fly. This is possible for the $(a, b, 0)$ class of distributions, defined by the recurrence $p_k = p_{k-1}(a + b/k)$ for constants a and b .

3.2 ITM for Discrete (Lattice) Distributions IV

The $(a, b, 0)$ Class and the ITR Algorithm

Three major count distributions belong to this class:

- **Binomial** $B(n, p)$: $p_0 = (1 - p)^n$, ratio $c_{k+1} = \frac{p(n - k)}{(1 - p)(k + 1)}$ for $k = 0, \dots, n - 1$. (n = number of trials, p = success probability.)
- **Poisson** $\text{Poi}(\lambda)$: $p_0 = e^{-\lambda}$, ratio $c_{k+1} = \frac{\lambda}{k + 1}$ for $k = 0, 1, \dots$ (λ (lambda) = rate parameter.)
- **Negative Binomial** $\text{NB}(r, p)$: $p_0 = (1 - p)^r$, ratio $c_{k+1} = \frac{(r + k)p}{k + 1}$ for $k = 0, 1, \dots$

3.2 ITM for Discrete (Lattice) Distributions V

Algorithm 10 (ITR): Generate $X \sim (p_k)$ via Ratio Recursion

Require: The starting probability p_0 and the ratios $c_{k+1} = p_{k+1}/p_k$ for $k = 0, 1, \dots$

- 1 Set $S \leftarrow p_0$; $P \leftarrow p_0$; $X \leftarrow 0$.
- 2 Generate $U \sim \mathcal{U}[0, 1)$.
- 3 **While** $U > S$: set $X \leftarrow X + 1$; $P \leftarrow P \cdot c_X$; $S \leftarrow S + P$.
- 4 Return X .

At each step, P holds the current probability p_X , obtained by multiplying the previous probability by the ratio c_X . This avoids pre-computing and storing the full PMF; we only ever hold two numbers (S and P) in memory.

Special case: Geometric Distribution $\text{Geo}(p)$. The geometric distribution models the number of failures before the first success, with PMF $p_k = (1 - p)^k \cdot p$ for $k = 0, 1, \dots$ and mean

3.2 ITM for Discrete (Lattice) Distributions VI

$\mathbb{E}[X] = (1 - p)/p$. Here p is the success probability and $q = 1 - p$ is the failure probability. For the geometric distribution, a closed-form inverse exists, bypassing the loop:

$$X = \lfloor \log U / \log(1 - p) \rfloor \sim \text{Geo}(p).$$

Proof. The survival function of $\text{Geo}(p)$ is $\mathbb{P}(X \geq k) = (1 - p)^k$. Setting $U \leq (1 - p)^k$ is equivalent to $\log U \leq k \log(1 - p)$ (since $\log(1 - p) < 0$, dividing reverses the inequality), giving $k \leq \log U / \log(1 - p)$. The floor function picks the largest such k .

3.2 ITM for Discrete (Lattice) Distributions VII

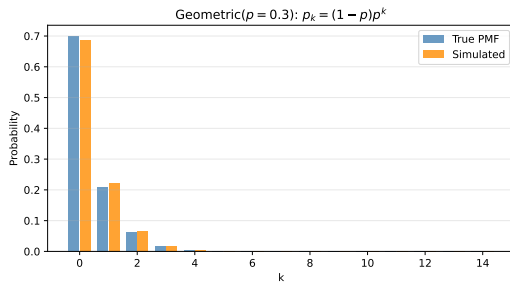


Figure: Histogram of 10 000 draws from $\text{Geo}(p = 0.3)$ via the closed-form ITM formula $X = \lfloor \log U / \log(0.7) \rfloor$, compared with the theoretical PMF (dots). The match is excellent.

Outline

- 1 3.1 Inverse Transform Method (ITM)
- 2 3.2 ITM Variants for Lattice Distributions
- 3 3.3 Ad Hoc Methods**
- 4 3.4 Uniform Distribution on Geometric Shapes
- 5 3.5 Acceptance-Rejection Method
- 6 3.6 Composition Method
- 7 3.7 Generating Gaussian Random Numbers
- 8 3.8 Uniform Distribution on Balls, Spheres, Ellipsoids

3.3.1 Binomial Distribution $B(n, p)$ I

The binomial distribution models the total number of successes in n independent *Bernoulli trials*, where each trial succeeds with probability p and fails with probability $1 - p$. The most natural simulation uses the defining structure directly: generate each trial individually and count the successes.

Concretely, if we define $\xi_j = 1$ (success) or $\xi_j = 0$ (failure) for trial j , then $X = \xi_1 + \xi_2 + \cdots + \xi_n$ counts the total successes. Each ξ_j can be simulated by drawing $U_j \sim \mathcal{U}[0, 1)$ and setting $\xi_j = 1$ if $U_j \leq p$, and $\xi_j = 0$ otherwise (since $\mathbb{P}(U_j \leq p) = p$).

3.3.1 Binomial Distribution $B(n, p)$ II

Algorithm 11 (DB): Generate $X \sim B(n, p)$

Require: Success probability $p \in (0, 1)$ and number of trials $n \geq 1$.

- 1 Set $X \leftarrow 0$.
- 2 For $i = 1$ to n :
 - Generate $U \sim \mathcal{U}[0, 1)$.
 - If $U \leq p$: set $X \leftarrow X + 1$ (record a success).
- 3 Return X .

This algorithm uses exactly n uniform random numbers and performs n comparisons. The expected output is $\mathbb{E}[X] = np$ and the variance is $\text{Var}(X) = np(1 - p)$. For large n with moderate mean $\lambda = np$, the ITR algorithm (Algorithm 10) is asymptotically faster, since its expected cost is $O(\lambda)$ rather than $O(n)$.

3.3.1 Binomial Distribution $B(n, p)$ III

Worked Numerical Example

Let $n = 5$ and $p = 0.4$. Draw five uniforms: $U_1 = 0.72$, $U_2 = 0.15$, $U_3 = 0.63$, $U_4 = 0.38$, $U_5 = 0.81$. Compare each to the threshold $p = 0.4$:

- $U_1 = 0.72 > 0.4$: failure.
- $U_2 = 0.15 \leq 0.4$: **success** (X becomes 1).
- $U_3 = 0.63 > 0.4$: failure.
- $U_4 = 0.38 \leq 0.4$: **success** (X becomes 2).
- $U_5 = 0.81 > 0.4$: failure.

Result: $X = 2$. The theoretical mean is $np = 5 \times 0.4 = 2.0$, so this particular draw returned exactly the mean.

3.3.2 Erlang Distribution $\text{Erl}(n, \lambda)$ I

The Erlang distribution models the total waiting time until the n -th event in a Poisson process with rate λ . For example, if customers arrive at rate $\lambda = 3$ per minute, the time until the 5th customer arrives follows $\text{Erl}(5, 3)$. Since each inter-arrival time is independently $\text{Exp}(\lambda)$, the Erlang distribution is simply the sum of n independent exponentials.

Definition and Simulation of $\text{Erl}(n, \lambda)$

Let $Y_1, Y_2, \dots, Y_n$ be independent, each $\sim \text{Exp}(\lambda)$. Then the sum $X = Y_1 + Y_2 + \dots + Y_n$ has the Erlang distribution $\text{Erl}(n, \lambda)$.

Using the ITM formula $Y_i = -\log(U_i)/\lambda$ for each exponential, and the logarithm property $\log(ab) = \log a + \log b$, we get the formula:

$$X = -\frac{1}{\lambda}(\log U_1 + \log U_2 + \dots + \log U_n) = -\frac{\log(U_1 \cdot U_2 \cdots U_n)}{\lambda},$$

where $U_1, \dots, U_n$ are independent $\mathcal{U}[0, 1)$ random variables.

3.3.2 Erlang Distribution $\text{Erl}(n, \lambda)$ II

Notation for the Erlang Simulation Formula

- n : the number of exponential stages (a positive integer).
- λ (lambda) > 0 : the rate of each exponential stage.
- $\log(\cdot)$: the natural logarithm (base $e \approx 2.718$).
- $U_1 \cdot U_2 \cdots U_n$: the product of all n uniform draws. This product lies in $(0, 1)$, so its logarithm is negative, and the negative sign at the front makes X positive.
- Mean: $\mathbb{E}[X] = n/\lambda$; Variance: $\text{Var}(X) = n/\lambda^2$.

Important numerical note. When n is large, computing the product $U_1 \cdots U_n$ directly risks *underflow*: with 64-bit floating-point numbers, the product of 1000 values each around 0.5 underflows to zero. Always accumulate the *sum of logarithms* $\log U_1 + \log U_2 + \cdots + \log U_n$ instead.

3.3.2 Erlang Distribution $\text{Erl}(n, \lambda)$ III

The survival function of $\text{Erl}(n, \lambda)$ has an elegant closed-form expression that we will use in the next section to derive a Poisson sampler:

$$\bar{F}(x) = \mathbb{P}(X > x) = e^{-\lambda x} \left(1 + \lambda x + \frac{(\lambda x)^2}{2!} + \cdots + \frac{(\lambda x)^{n-1}}{(n-1)!} \right).$$

This is a partial sum of the Taylor series of $e^{\lambda x}$, multiplied by $e^{-\lambda x}$.

The probability density function of $\text{Erl}(n, \lambda)$ is $f(x) = \frac{\lambda^n}{\Gamma(n)} x^{n-1} e^{-\lambda x}$ for $x \geq 0$, where $\Gamma(n) = (n-1)!$ for integer n . This is a special case of the Gamma distribution $\text{Gamma}(n, \lambda)$.

3.3.2 Erlang Distribution $\text{Erl}(n, \lambda)$ IV

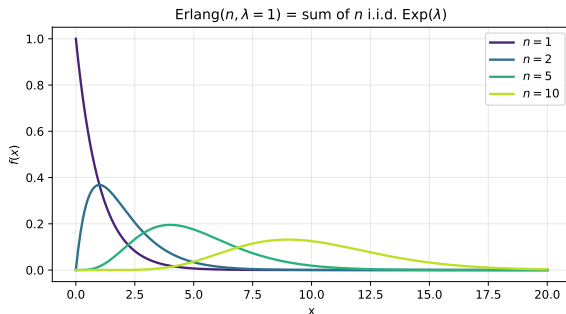


Figure: Erlang densities for $\text{Erl}(n, 1)$ with $n = 1, 2, 3, 5$. When $n = 1$ the Erlang is simply $\text{Exp}(1)$ (exponential). As n increases, the distribution becomes more symmetric and concentrates closer to its mean $n/\lambda = n$, reflecting the averaging effect of summing many independent exponentials.

3.3.3 Poisson Distribution $\text{Poi}(\lambda)$ I

The Poisson distribution counts the number of events that occur in a fixed time window $[0, 1]$ when events arrive at rate $\lambda > 0$ independently of each other. We derive a beautiful simulation algorithm from the *renewal process* interpretation of the Poisson process.

Idea. Imagine inter-arrival times $\tau_1, \tau_2, \dots$ each drawn independently from $\text{Exp}(1)$, representing the gaps between successive events. The partial sum $\sigma_k = \tau_1 + \tau_2 + \dots + \tau_k$ is the time of the k -th event. The number of events in the window $(0, \lambda]$ is:

$$N = \begin{cases} \max\{i \geq 1 : \tau_1 + \dots + \tau_i \leq \lambda\} & \text{if } \tau_1 \leq \lambda, \\ 0 & \text{if } \tau_1 > \lambda. \end{cases}$$

Proposition 3.3.1 — Poisson via Renewal Process

N (defined above) has the Poisson distribution: $N \sim \text{Poi}(\lambda)$.

3.3.3 Poisson Distribution $\text{Poi}(\lambda)$ II

Proof sketch. Using the Erlang survival formula from the previous slide:

$\mathbb{P}(N \leq k) = \mathbb{P}(\sigma_{k+1} > \lambda) = e^{-\lambda}(1 + \lambda + \cdots + \lambda^k/k!)$. The PMF then follows:

$$\mathbb{P}(N = k) = \mathbb{P}(N \leq k) - \mathbb{P}(N \leq k - 1) = e^{-\lambda} \frac{\lambda^k}{k!},$$

which is exactly the Poisson PMF.

3.3.3 Poisson Distribution $\text{Poi}(\lambda)$ III

Substituting $\tau_i = -\log U_i$ (the ITM formula for $\text{Exp}(1)$) transforms the stopping criterion $\sum_{i=1}^n \tau_i \leq \lambda$ into an equivalent criterion about products:

$$\tau_1 + \cdots + \tau_n \leq \lambda \iff -(\log U_1 + \cdots + \log U_n) \leq \lambda \iff \log(U_1 \cdots U_n) \geq -\lambda \iff \prod_{i=1}^n U_i \geq e^{-\lambda}.$$

So we count how many uniform numbers we can multiply before the running product falls below $e^{-\lambda}$ ($\lambda = \text{rate parameter}$).

3.3.3 Poisson Distribution $\text{Poi}(\lambda)$ IV

Algorithm 12 (DP): Generate $X \sim \text{Poi}(\lambda)$

Require: Rate parameter $\lambda > 0$.

- 1 Set $X \leftarrow 0$; generate $U \sim \mathcal{U}[0, 1)$; set $S \leftarrow -\log U$.
- 2 **While** $S < \lambda$:
 - Generate $U \sim \mathcal{U}[0, 1)$; set $S \leftarrow S - \log U$; set $X \leftarrow X + 1$.
- 3 **Return** X .

The variable S accumulates the sum $-\log U_1 - \log U_2 - \dots$, which equals $\tau_1 + \tau_2 + \dots$ (a sum of $\text{Exp}(1)$ inter-arrivals). We stop when the total time first exceeds λ . The expected number of uniform random numbers consumed is $\mathbb{E}[X] + 1 = \lambda + 1$.

3.3.3 Poisson Distribution $\text{Poi}(\lambda)$ V

Worked Numerical Example

Let $\lambda = 2$. Draw $U_1 = 0.7$, $U_2 = 0.5$, $U_3 = 0.3$.

- ① $S = -\log(0.7) \approx 0.357$. Is $S < 2$? Yes. $X = 1$.
- ② $S = 0.357 + (-\log(0.5)) = 0.357 + 0.693 = 1.050$. Is $S < 2$? Yes. $X = 2$.
- ③ $S = 1.050 + (-\log(0.3)) = 1.050 + 1.204 = 2.254$. Is $S < 2$? No. Stop.

Result: $X = 2$. The theoretical mean is $\lambda = 2$.

3.3.3 Poisson Distribution $\text{Poi}(\lambda)$ VI

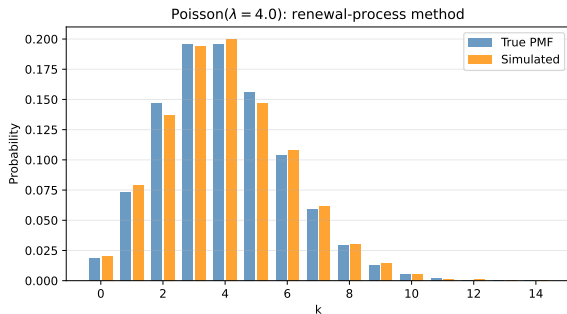


Figure: Histogram of 10 000 samples from $\text{Poi}(3)$ generated via Algorithm 12 (DP), compared with the true PMF (dots). The empirical frequencies closely match the theoretical probabilities.

Key Insight: Poisson as a Counting Problem

Algorithm 12 gives a beautiful probabilistic interpretation: we are watching a Poisson process unfold in real time. Each $-\log U_i$ is an $\text{Exp}(1)$ inter-arrival time. We count how many arrivals (X events) fit before the total elapsed time S exceeds the window length λ . This connects simulation directly to the underlying physical model of the Poisson process.

3.3 Python: Poisson and Erlang Simulation

Listing 1: Ad hoc methods for Poisson and Erlang distributions (Algorithms 12 and Erlang sum)

```
1 import numpy as np
2 rng = np.random.default_rng(0)
3
4 def sample_poisson(lam, rng):
5     """Renewal-process algorithm (Alg. 12) for Poi(lam).
6     Accumulate Exp(1) inter-arrivals until total exceeds lam."""
7     U = rng.uniform()
8     S = -np.log(U)    # first Exp(1) inter-arrival time
9     X = 0
10    while S < lam:
11        U = rng.uniform()
12        S -= np.log(U)    # add next Exp(1) inter-arrival
13        X += 1
14    return X
15
16 def sample_erlang(n, lam, rng):
17     """Sum of n Exp(lam) variables => Erl(n, lam).
18     Numerically stable: accumulate logs, never multiply."""
19     logs = rng.uniform(0, 1, n)
20     return -np.log(logs).sum() / lam
21
22 # Verify: empirical vs theoretical moments
23 samples_poi = [sample_poisson(3.5, rng) for _ in range(10000)]
24 samples_erl = [sample_erlang(4, 1.0, rng) for _ in range(10000)]
25 print(f"Poi(3.5) mean = {np.mean(samples_poi):.3f} (theory: 3.5)")
```

Outline

- 1 3.1 Inverse Transform Method (ITM)
- 2 3.2 ITM Variants for Lattice Distributions
- 3 3.3 Ad Hoc Methods
- 4 3.4 Uniform Distribution on Geometric Shapes**
- 5 3.5 Acceptance-Rejection Method
- 6 3.6 Composition Method
- 7 3.7 Generating Gaussian Random Numbers
- 8 3.8 Uniform Distribution on Balls, Spheres, Ellipsoids

3.4 Uniform Distribution on Subsets of $\mathbb{R}^d$ I

A random vector $X \in A \subset \mathbb{R}^d$ has the *uniform distribution* $\mathcal{U}(A)$ if it is equally likely to fall anywhere in A . Formally, for any measurable subset $D \subset A$,

$$\mathbb{P}(X \in D) = \frac{\text{Vol } D}{\text{Vol } A},$$

where Vol denotes the d -dimensional volume (length for $d = 1$, area for $d = 2$, ordinary volume for $d = 3$). Equivalently, the density is $f(x) = \frac{1}{\text{Vol } A} 1(x \in A)$, where $1(\cdot)$ is the indicator function (1 if $x \in A$, else 0).

Rectangular regions. Sampling uniformly in the rectangle $[a_1, b_1] \times \cdots \times [a_d, b_d]$ is straightforward: generate d independent uniforms $U_1, \dots, U_d \sim \mathcal{U}[0, 1]$ and apply a linear rescaling: $V_i = (b_i - a_i)U_i + a_i$ for $i = 1, \dots, d$. The output $V = (V_1, \dots, V_d)$ is then uniform in the rectangle. For non-rectangular regions, a rejection approach based on the following proposition works well in low dimensions.

3.4 Uniform Distribution on Subsets of $\mathbb{R}^d$ II

Proposition 3.4.1 — Uniform on a Subset via Rejection

Let $A \subset B$ be measurable sets with $\text{Vol}(B) < \infty$. If $X \sim \mathcal{U}(B)$ (uniform in the larger set B), then the conditional distribution $(X \mid X \in A)$ is $\mathcal{U}(A)$.

Proof. For any $D \subset A$, by the definition of conditional probability:

$$\mathbb{P}(X \in D \mid X \in A) = \frac{\mathbb{P}(X \in D)}{\mathbb{P}(X \in A)} = \frac{\text{Vol } D / \text{Vol } B}{\text{Vol } A / \text{Vol } B} = \frac{\text{Vol } D}{\text{Vol } A}.$$

This is the uniform distribution on A .

3.4 Uniform Distribution on Subsets of $\mathbb{R}^d$ III

Algorithm 13 — Rejection Sampling for $\mathcal{U}(A)$

Require: A bounding rectangle $B = [a_1, b_1] \times \cdots \times [a_d, b_d]$ containing A , and a way to test whether a point V lies in A .

- 1 Generate $U_1, \dots, U_d$ i.i.d. $\mathcal{U}[0, 1]$; set $V_i = (b_i - a_i)U_i + a_i$ for each $i = 1, \dots, d$.
- 2 If $V = (V_1, \dots, V_d) \in A$: set $X = V$ and return X .
- 3 Otherwise go back to step 1 and try again.

Each trial is accepted with probability $\text{Vol}(A)/\text{Vol}(B)$. The expected number of trials is $\text{Vol}(B)/\text{Vol}(A)$.

The curse of dimensionality for d -dimensional balls. The d -dimensional unit ball $B_d = \{x \in \mathbb{R}^d : \|x\| \leq 1\}$ has volume $\text{Vol}(B_d) = \pi^{d/2}/\Gamma(d/2 + 1)$, where Γ (Gamma) is the generalization of the factorial function. The enclosing hypercube $[-1, 1]^d$ has volume 2^d . Their ratio is:

$$\frac{\text{Vol}(B_d)}{\text{Vol}([-1, 1]^d)} = \frac{\pi^{d/2}}{2^d \Gamma(d/2 + 1)} \xrightarrow{d \rightarrow \infty} 0$$

3.4 Uniform Distribution on Subsets of $\mathbb{R}^d$ IV

exponentially fast.

How Bad Is the Curse?

At $d = 2$: acceptance probability $= \pi/4 \approx 78.5\%$ (quite good). At $d = 10$: acceptance probability $\approx 0.25\%$ (1 in 400). At $d = 30$: acceptance probability $\approx 2 \times 10^{-14}$ (1 in 5×10^{13}). Algorithm 13 is therefore completely impractical for high-dimensional balls. Section 3.8 provides the correct approach using the Normal distribution trick.

Outline

- 1 3.1 Inverse Transform Method (ITM)
- 2 3.2 ITM Variants for Lattice Distributions
- 3 3.3 Ad Hoc Methods
- 4 3.4 Uniform Distribution on Geometric Shapes
- 5 3.5 Acceptance-Rejection Method**
- 6 3.6 Composition Method
- 7 3.7 Generating Gaussian Random Numbers
- 8 3.8 Uniform Distribution on Balls, Spheres, Ellipsoids

3.5.1 AR-d: Discrete Acceptance-Rejection I

The acceptance-rejection (AR) method, introduced by John von Neumann, is the most adaptable general-purpose sampling technique. It applies whenever we want to sample from a target distribution (p_j) but can only easily sample from a “proposal” distribution (q_j). The method requires a finite constant $c \geq 1$ such that $p_j \leq c q_j$ for every j in the state space.

Intuition. Think of q_j as a budget of probability mass allocated to value j . The target p_j is at most $c q_j$, so the ratio $p_j/(c q_j) \in [0, 1]$ measures what fraction of the budget at j should “count.” We propose a value $Y = j$ from the easy distribution, then accept it with probability $p_j/(c q_j)$. If accepted, the output $X = j$ has the correct distribution (p_j); if rejected, we try again.

Proposition 3.5.1 — AR-d Correctness

Let $U \sim \mathcal{U}[0, 1]$ and $Y \sim (q_j)$ be independent. Define the acceptance event $A = \{c U q_Y \leq p_Y\}$. Then the acceptance probability and conditional distribution are:

$$\mathbb{P}(A) = \frac{1}{c} \quad \text{and} \quad \mathbb{P}(Y = j \mid A) = p_j.$$

3.5.1 AR-d: Discrete Acceptance-Rejection II

Notation for Proposition 3.5.1

- c : the *bounding constant*, satisfying $\sup_j p_j/q_j \leq c$. A smaller c means higher acceptance probability $1/c$, so we want c as small as possible.
- A : the event that the proposed value Y is accepted. Acceptance happens when $U \leq p_Y/(c q_Y)$, i.e. when the uniform draw falls below the ratio of the target probability to the proposal times c .
- $\mathbb{P}(A) = 1/c$: on average, we need c proposals per accepted sample.

Proof. By conditioning on the proposed value $Y = j$:

$$\mathbb{P}(Y = j, A) = q_j \cdot \mathbb{P}\left(U \leq \frac{p_j}{c q_j}\right) = q_j \cdot \frac{p_j}{c q_j} = \frac{p_j}{c}.$$

Summing over all j : $\mathbb{P}(A) = \sum_j p_j/c = 1/c$. Finally,
 $\mathbb{P}(Y = j | A) = \mathbb{P}(Y = j, A)/\mathbb{P}(A) = (p_j/c)/(1/c) = p_j$.

3.5.1 AR-d: Discrete Acceptance-Rejection III

Algorithm 14 (AR-d): Generate $X \sim (p_j)$

Require: Bounding constant c with $p_j \leq cq_j$ for all j ; an algorithm for sampling $Y \sim (q_j)$.

➊ **Repeat:**

- Generate $Y \sim (q_j)$ and independently $U \sim \mathcal{U}[0, 1)$.

➋ **Until** $c U q_Y \leq p_Y$ (acceptance test).

➌ **Return** $X = Y$.

The number of loop iterations L has a geometric distribution: $\mathbb{P}(L = k) = (1 - 1/c)^{k-1} \cdot (1/c)$, with mean $\mathbb{E}[L] = c$. A smaller c means fewer iterations. Therefore always choose the *smallest* possible c , i.e. $c = \sup_j p_j/q_j$.

3.5.1 AR-d: Discrete Acceptance-Rejection IV

Example 3.5.2 — Sampling from the Zeta Distribution

We want to sample X with PMF $p_j = \frac{6}{\pi^2 j^2}$ for $j = 1, 2, \dots$ (proportional to $1/j^2$; normalised by $\sum_{j=1}^{\infty} 1/j^2 = \pi^2/6$).

Proposal: $q_j = 1/(j(j+1))$ for $j = 1, 2, \dots$. This is easy to sample: note $1/(j(j+1)) = 1/j - 1/(j+1)$, so $F_q(k) = 1 - 1/(k+1)$, and $Y = \lfloor 1/(1-U) \rfloor$ generates $Y \sim (q_j)$.

Bounding constant: The ratio $p_j/q_j = \frac{6}{\pi^2 j^2} \cdot j(j+1) = \frac{6(j+1)}{\pi^2 j}$, which is maximized at $j = 1$ giving $c = 12/\pi^2 \approx 1.216$.

Acceptance condition: $cUq_Y \leq p_Y$ simplifies to $2UY \leq Y + 1$. The acceptance rate is $1/c = \pi^2/12 \approx 82.2\%$.

3.5.2 AR-c: Continuous Acceptance-Rejection I

The continuous version of the AR method targets a continuous density $f(x)$ on a set $E \subseteq \mathbb{R}^d$, using a proposal density $g(x)$. The requirement is the same: there must exist a finite constant c such that $f(x) \leq c g(x)$ for all $x \in E$. This means the scaled proposal density $c g(x)$ forms an *envelope* above the target density everywhere.

Proposition 3.5.3 — AR-c Correctness

Let $Y \sim g$ and $U \sim \mathcal{U}[0, 1)$ be independent (that is, Y is drawn from the proposal density g , and U is an independent uniform). Define the acceptance event $A = \{c U g(Y) \leq f(Y)\}$. Then the acceptance probability and the conditional distribution of Y given acceptance are:

$$\mathbb{P}(A) = \frac{1}{c} \quad \text{and} \quad \mathbb{P}(Y \in B \mid A) = \int_B f(y) dy \quad \text{for all } B \subset E.$$

3.5.2 AR-c: Continuous Acceptance-Rejection II

Proof. The joint density of (U, Y) is $g(y) du dy$ on $[0, 1] \times E$. Restricting to the acceptance region $\{(u, y) : c u g(y) \leq f(y)\}$:

$$\mathbb{P}(Y \in B, A) = \int_B \int_0^{f(y)/(c g(y))} g(y) du dy = \int_B \frac{f(y)}{c} dy.$$

Dividing by $\mathbb{P}(A) = 1/c$ gives $\mathbb{P}(Y \in B \mid A) = \int_B f(y) dy$.

Algorithm 16 (AR-c): Generate $X \sim f$

Require: Constant c with $f(x) \leq c g(x)$ for all x ; an algorithm for sampling $Y \sim g$.

- ➊ **Repeat:** generate $Y \sim g$ and independently $U \sim \mathcal{U}[0, 1]$.
- ➋ **Until** $c U g(Y) \leq f(Y)$.
- ➌ **Return** $X = Y$.

3.5.2 AR-c: Continuous Acceptance-Rejection III

Example 3.5.4 — Generating $\mathcal{N}(0, 1)$ via AR-c (Algorithm AR-N).

The standard normal density is $\phi(x) = \frac{1}{\sqrt{2\pi}}e^{-x^2/2}$ (ϕ , the Greek letter). By symmetry, we first generate the absolute value $|Z|$ from the *half-normal* density $f(x) = \frac{2}{\sqrt{2\pi}}e^{-x^2/2}$ for $x > 0$, then randomly negate it.

We use the $\text{Exp}(1)$ density $g(x) = e^{-x}$ as the proposal, because it is easy to sample via ITM and has a shape similar to f on $(0, \infty)$.

The ratio $f(x)/g(x) = \frac{2}{\sqrt{2\pi}}e^{x-x^2/2}$. To find the bounding constant, maximize the exponent $h(x) = x - x^2/2$: set $h'(x) = 1 - x = 0$, giving $x^* = 1$, $h(1) = 1/2$. Therefore:

$$c = \frac{2}{\sqrt{2\pi}}e^{1/2} = \sqrt{\frac{2e}{\pi}} \approx 1.315.$$

The acceptance rate is $1/c = \sqrt{\pi/(2e)} \approx 76.0\%$.

The acceptance condition $cUg(Y) \leq f(Y)$ simplifies (dividing both sides by $g(Y) = e^{-Y}$) to:

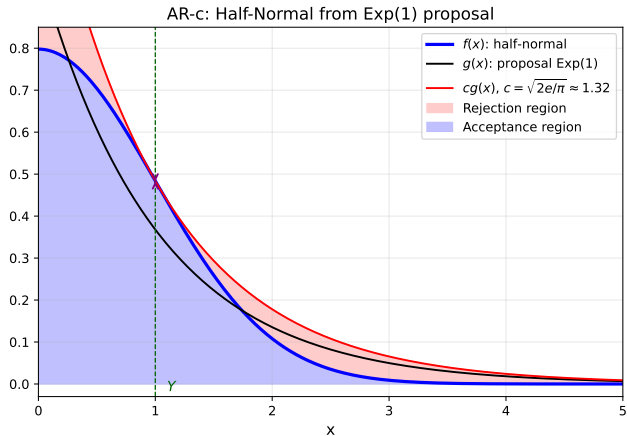
$$U \leq \exp\left(-\frac{(Y-1)^2}{2}\right).$$

3.5.2 AR-c: Continuous Acceptance-Rejection IV

Algorithm 17 (AR-N): Generate $Z \sim \mathcal{N}(0, 1)$

- 1 Generate $U_1 \sim \mathcal{U}[0, 1)$; set $Y = -\log(U_1)$. (This gives $Y \sim \text{Exp}(1)$ via ITM.)
- 2 Generate $U_2 \sim \mathcal{U}[0, 1)$ independently.
- 3 If $U_2 > \exp(-(Y - 1)^2/2)$: the proposal is rejected; go back to step 1.
- 4 Generate $U_3 \sim \mathcal{U}[0, 1)$; set $I = 2\lfloor 2U_3 \rfloor - 1$ (so $I = +1$ or $I = -1$ with equal probability $1/2$).
- 5 Return $Z = I \cdot Y$.

3.5.2 AR-c: Continuous Acceptance-Rejection V



3.5.2 AR-c: Continuous Acceptance-Rejection VI

Figure: The half-normal target density $f(x) = \sqrt{2/\pi} e^{-x^2/2}$ (blue), the proposal density $g(x) = e^{-x}$ of $\text{Exp}(1)$ (gray), and the envelope $c \cdot g(x)$ (dashed orange). A proposed point $(Y, U \cdot c \cdot g(Y))$ is accepted when it falls in the blue region, i.e. below the target density f .

3.5.2 AR-c: Continuous Acceptance-Rejection VII

Unknown normalising constants. In many practical settings (e.g. Bayesian inference), we only know the unnormalised densities $f^*(x)$ and $g^*(x)$, related to the actual densities by $f(x) = f^*(x)/c_f$ and $g(x) = g^*(x)/c_g$ for unknown constants c_f and c_g . In this case, compute $c^* = \sup_x f^*(x)/g^*(x)$. The acceptance condition $cUg(Y) \leq f(Y)$ then becomes $c^*Ug^*(Y) \leq f^*(Y)$, which does not require knowing c_f or c_g . This makes AR-c especially powerful for Bayesian posterior sampling.

Example 3.5.5 — Gamma(α, λ) for α (alpha) < 1 (Madras). When $\alpha < 1$, the Gamma density $f(x) \propto x^{\alpha-1}e^{-\lambda x}$ is unbounded near $x = 0$. The Madras algorithm uses a two-component mixture proposal: a power-law density $g_1(x) \propto x^{\alpha-1}$ on $(0, 1)$, and an exponential density $g_2(x) = e^{-x}$ on $[1, \infty)$. The bounding constant is $c = 1/(K \Gamma(\alpha))$ where $K = \alpha e/(\alpha + e)$.

Example 3.5.6 — Central χ_d^2 Distribution. The chi-squared distribution with d degrees of freedom is $\chi_d^2 = Z_1^2 + \dots + Z_d^2$, the sum of d squared standard normals. Since $\chi_d^2 \sim \text{Gamma}(d/2, 1/2)$, and for even $d = 2k$ the Erlang formula applies: $\chi_{2k}^2 \sim \text{Erl}(k, 1/2)$, which can be generated directly.

3.5 Python: AR-c for $\mathcal{N}(0, 1)$

Listing 2: AR-N (Algorithm 17): generate standard normal via acceptance-rejection with $\text{Exp}(1)$ proposal

```
1 import numpy as np
2 rng = np.random.default_rng(42)
3
4 def sample_normal_arn(n, rng):
5     """
6     Algorithm 17 AR-N: generate  $N(0,1)$  via AR-c.
7     Target: half-normal  $f(x) = \sqrt{2/\pi} \exp(-x^2/2)$ ,  $x > 0$ .
8     Proposal:  $\text{Exp}(1)$ ,  $g(x) = \exp(-x)$ ,  $x > 0$ .
9     Bounding constant  $c = \sqrt{2e/\pi} \sim 1.315$ .
10    Acceptance rate:  $1/c = \sqrt{\pi/(2e)} \sim 76.0\%$ .
11    """
12    out = np.empty(n)
13    i = 0
14    total_trials = 0
15    while i < n:
16        U1 = rng.uniform()
17        Y = -np.log(U1) # Y ~ Exp(1) via ITM
18        U2 = rng.uniform()
19        total_trials += 1
20        if U2 <= np.exp(-(Y - 1)**2 / 2): # acceptance test
21            sign = 1 if rng.uniform() < 0.5 else -1
22            out[i] = sign * Y
23            i += 1
24    print(f"Acceptance rate: {n/total_trials:.3f} ")
25    print(f"(theory: {np.sqrt(np.pi/(2*np.e)):.3f})")
```

Outline

- 1 3.1 Inverse Transform Method (ITM)
- 2 3.2 ITM Variants for Lattice Distributions
- 3 3.3 Ad Hoc Methods
- 4 3.4 Uniform Distribution on Geometric Shapes
- 5 3.5 Acceptance-Rejection Method
- 6 3.6 Composition Method**
- 7 3.7 Generating Gaussian Random Numbers
- 8 3.8 Uniform Distribution on Balls, Spheres, Ellipsoids

3.6 Composition (Mixture) Method I

The composition method addresses the case where the target distribution is a *mixture*: its density (or PMF) can be written as a weighted combination (a convex combination) of simpler component densities that we can already sample efficiently. If we know the component proportions and can sample each component, then sampling the mixture is essentially free.

3.6 Composition (Mixture) Method II

Composition Method: Setup and Algorithm

Suppose the target density has the mixture form

$$f(x) = \sum_{j=1}^n p_j g_j(x),$$

where:

- each g_j is a valid probability density function (non-negative, integrates to 1),
- $(p_j)_{j=1}^n$ is a probability vector: $p_j \geq 0$ and $\sum_j p_j = 1$,
- we can sample from each component density g_j efficiently.

Algorithm (Composition):

- 1 Sample a component index J with $\mathbb{P}(J = j) = p_j$ (this is itself a discrete sampling problem, solvable by ITM-d or a table).
- 2 Conditional on $J = j$, sample $X \sim g_j$ from component j .
- 3 Return X .

The output X has the mixture density f , because for any measurable set B :

3.6 Python: Mixture Sampling

Listing 3: Composition method for a Gaussian mixture and NB via Gamma-Poisson hierarchy

```
1 import numpy as np
2 rng = np.random.default_rng(0)
3
4 def sample_mixture(weights, mus, sigmas, n, rng):
5     """
6     Composition method for a Gaussian mixture.
7     weights: mixture probabilities summing to 1.
8     mus, sigmas: mean and std of each component.
9     Step 1: sample component index  $J \sim \text{Categorical}(\text{weights})$ .
10    Step 2: sample  $X$  from component  $J$ .
11    """
12    weights = np.array(weights)
13    J = rng.choice(len(weights), size=n, p=weights)
14    X = np.array([rng.normal(mus[j], sigmas[j]) for j in J])
15    return X
16
17 def sample_nb_gamma_poisson(alpha, lam, n, rng):
18     """
19     Sample NB(alpha, 1/(1+lam)) via Gamma-Poisson hierarchy.
20     Step 1:  $\text{Lambda} \sim \text{Gamma}(\text{alpha}, \text{scale}=1/\text{lam})$ .
21     Step 2:  $X \mid \text{Lambda} \sim \text{Poi}(\text{Lambda})$ .
22     """
23    Lambda = rng.gamma(alpha, 1.0/lam, size=n)
24    X = rng.poisson(Lambda)
25    return X
```

Outline

- 1 3.1 Inverse Transform Method (ITM)
- 2 3.2 ITM Variants for Lattice Distributions
- 3 3.3 Ad Hoc Methods
- 4 3.4 Uniform Distribution on Geometric Shapes
- 5 3.5 Acceptance-Rejection Method
- 6 3.6 Composition Method
- 7 3.7 Generating Gaussian Random Numbers**
- 8 3.8 Uniform Distribution on Balls, Spheres, Ellipsoids

3.7.1 Box-Muller Method I

The standard normal distribution $\mathcal{N}(0, 1)$ is the most important distribution in statistics and simulation. We present three methods for generating standard normal samples; the first is the Box-Muller transform, which generates *two* independent $\mathcal{N}(0, 1)$ variables at once using polar coordinates. The key structural observation is that the joint density of two independent standard normals has a special factorization in polar coordinates.

Lemma 3.7.1 — Polar Structure of Bivariate Normal

Let $Z_1, Z_2 \stackrel{\text{i.i.d.}}{\sim} \mathcal{N}(0, 1)$ and let (D, Θ) be their polar coordinates: $Z_1 = D \cos \Theta$, $Z_2 = D \sin \Theta$, where $D = \sqrt{Z_1^2 + Z_2^2}$ is the radius and $\Theta \in [0, 2\pi)$ is the angle. Then D and Θ are *independent*, with:

- $\Theta \sim \mathcal{U}[0, 2\pi)$ (the angle is uniformly distributed).
- D has the **Rayleigh** density $f_D(r) = r e^{-r^2/2}$ for $r > 0$.

Proof. The joint density of (Z_1, Z_2) is $f(z_1, z_2) = \frac{1}{2\pi} e^{-(z_1^2 + z_2^2)/2}$. Substituting the polar parametrisation $z_1 = r \cos \theta$, $z_2 = r \sin \theta$ with Jacobian r (the area element transforms as $dz_1 dz_2 = r dr d\theta$):

$$f_{(D, \Theta)}(r, \theta) = \frac{1}{2\pi} e^{-r^2/2} \cdot r = \underbrace{r e^{-r^2/2}}_{=f_D(r)} \cdot \underbrace{\frac{1}{2\pi}}_{=f_\Theta(\theta)}.$$

3.7.1 Box-Muller Method II

The density factors into a product of a function of r alone and a function of θ alone, which is the characterization of independence.

Lemma 3.7.2 — Simulating the Rayleigh Radius

If D has the Rayleigh density $f_D(r) = re^{-r^2/2}$, then $D^2 \sim \text{Exp}(1/2)$. For $U \sim \mathcal{U}[0, 1)$, the ITM formula gives:

$$D \stackrel{\mathcal{D}}{=} (-2 \log U)^{1/2}.$$

Proof. The survival function of D^2 is $\mathbb{P}(D^2 > x) = \int_{\sqrt{x}}^{\infty} re^{-r^2/2} dr = e^{-x/2}$, which is the survival function of $\text{Exp}(1/2)$.

3.7.1 Box-Muller Method III

Combining the two lemmas: we can simulate (D, Θ) using two uniforms — U_1 to get $D = (-2 \log U_1)^{1/2}$ via the Rayleigh ITM, and U_2 to get $\Theta = 2\pi U_2$ uniformly on $[0, 2\pi)$. Then convert back to Cartesian coordinates.

Proposition 3.7.3 — Box-Muller Transform

Let $U_1, U_2 \stackrel{\text{i.i.d.}}{\sim} \mathcal{U}[0, 1)$ and define:

$$Z_1 = (-2 \log U_1)^{1/2} \cos(2\pi U_2), \quad Z_2 = (-2 \log U_1)^{1/2} \sin(2\pi U_2).$$

Then Z_1 and Z_2 are independent $\mathcal{N}(0, 1)$ random variables.

3.7.1 Box-Muller Method IV

Notation for Box-Muller

- $(-2 \log U_1)^{1/2}$: the Rayleigh radius, derived from U_1 .
- $2\pi U_2$: the angle Θ (theta), uniform on $[0, 2\pi)$ (two pi times a uniform gives a random angle in radians).
- $\cos$ and $\sin$: standard trigonometric functions converting polar (D, Θ) back to Cartesian (Z_1, Z_2) .
- Two uniforms in, two independent normals out — very efficient.

3.7.1 Box-Muller Method V

Algorithm 20 (BM): Generate $Z_1, Z_2 \sim \mathcal{N}(0, 1)$

- 1 Generate U_1, U_2 i.i.d. $\mathcal{U}[0, 1)$.
- 2 Compute the radius: $D \leftarrow (-2 \log U_1)^{1/2}$.
- 3 Compute the angle: $V \leftarrow 2\pi U_2$.
- 4 Compute: $Z_1 \leftarrow D \cos V$ and $Z_2 \leftarrow D \sin V$.
- 5 Return (Z_1, Z_2) .

This algorithm produces *two* independent standard normal samples from *two* uniform random numbers, with no rejected samples.

3.7.1 Box-Muller Method VI

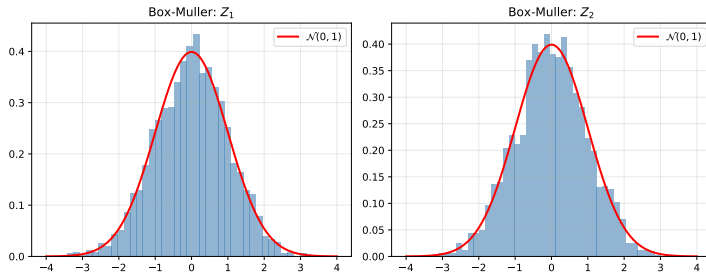


Figure: Box-Muller output: histograms of Z_1 (left panel) and Z_2 (right panel) from 5000 draws using Algorithm 20, overlaid with the standard normal density $\phi(z) = (2\pi)^{-1/2}e^{-z^2/2}$ (curve). Both histograms closely match the theoretical density.

3.7.1 Marsaglia-Bray Method I

The Box-Muller method requires computing $\cos$ and $\sin$, which are slow on older hardware (each involving many arithmetic operations via Taylor series or table lookups). Marsaglia and Bray (1964) proposed a variant that avoids all trigonometric functions by generating the polar angle implicitly from uniform random points inside the unit disc.

Key observation. If (W_1, W_2) is uniform on the unit disc B_2 , then its polar coordinates (D', Θ') satisfy $D'^2 \sim \mathcal{U}[0, 1)$ and $\Theta' \sim \mathcal{U}[0, 2\pi)$, and they are independent. Moreover, $\cos \Theta' = W_1/D'$ and $\sin \Theta' = W_2/D'$ are obtained without any trigonometric evaluation — we just normalize the vector (W_1, W_2) .

Proposition 3.7.4 — Marsaglia-Bray Transform

Let $V_1, V_2 \stackrel{\text{i.i.d.}}{\sim} \mathcal{U}[-1, 1)$ and set $X' = V_1^2 + V_2^2$. Conditional on $X' \leq 1$ (i.e. the point (V_1, V_2) falls inside the unit disc), define $Y = \sqrt{-2 \log(X')/X'}$ and set:

$$Z_1 = V_1 \cdot Y, \quad Z_2 = V_2 \cdot Y.$$

Then Z_1 and Z_2 are independent $\mathcal{N}(0, 1)$ random variables.

3.7.1 Marsaglia-Bray Method II

Notation for Marsaglia-Bray

- $V_1, V_2 \sim \mathcal{U}[-1, 1)$: uniform on $[-1, 1)$, generated as $V_i = 2U_i - 1$ from $U_i \sim \mathcal{U}[0, 1)$.
- $X' = V_1^2 + V_2^2$: the squared distance from the origin; we only proceed if $X' \leq 1$ (point is inside the unit disc).
- $Y = \sqrt{-2 \log(X')/X'}$: the Box-Muller radius $D = \sqrt{-2 \log(X')}$, divided by $D' = \sqrt{X'}$, giving the factor D/D' that converts Cartesian coordinates to the correctly scaled output.

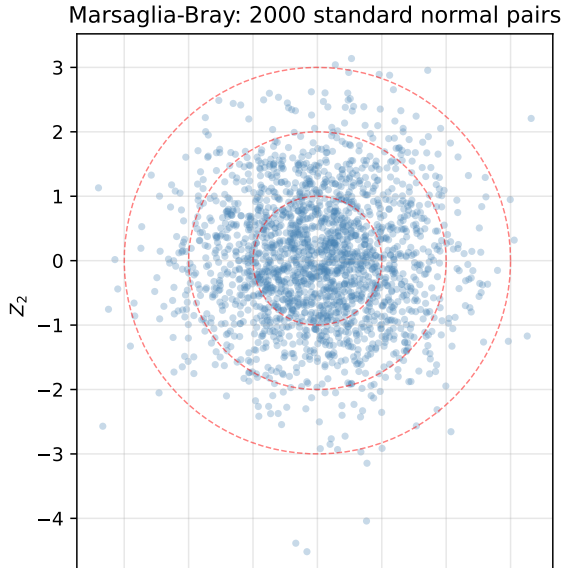
3.7.1 Marsaglia-Bray Method III

Algorithm 21 (MB): Generate $Z_1, Z_2 \sim \mathcal{N}(0, 1)$

- ➊ **Repeat:** generate U_1, U_2 i.i.d. $\mathcal{U}[0, 1)$; set $V_1 = 2U_1 - 1$, $V_2 = 2U_2 - 1$; compute $X' = V_1^2 + V_2^2$.
- ➋ **Until** $X' \leq 1$ (the point lies inside the unit disc).
- ➌ Compute $Y \leftarrow \sqrt{-2 \log(X')/X'}$.
- ➍ Compute $Z_1 \leftarrow V_1 Y$ and $Z_2 \leftarrow V_2 Y$.
- ➎ Return (Z_1, Z_2) .

The acceptance probability is $\mathbb{P}(V_1^2 + V_2^2 \leq 1) = \pi/4 \approx 78.5\%$ (the area of the unit disc divided by the area of the 2×2 square), so on average $4/\pi \approx 1.27$ pairs (V_1, V_2) are generated per accepted pair.

3.7.1 Marsaglia-Bray Method IV



Box-Muller vs. Marsaglia-Bray

Box-Muller: requires cos and sin (slow on older hardware); accepts every proposal (no rejections).

Marsaglia-Bray: no trigonometric functions; approximately 21.5% of proposals are rejected (outside the disc).

On modern CPUs with fast trig instructions, Box-Muller is competitive. On older hardware, embedded systems, or in compiled low-level code, Marsaglia-Bray is often faster. Both produce exact $\mathcal{N}(0, 1)$ samples (not approximations).

3.7.2 ITM for Normal Variables — Numerical Inversion of Φ I

To generate $Z \sim \mathcal{N}(0, 1)$ via the inverse transform method, we need the quantile function $\Phi^{-1}(u)$, where Φ (Phi, uppercase) is the standard normal c.d.f. $\Phi(x) = \mathbb{P}(Z \leq x) = \int_{-\infty}^x \frac{1}{\sqrt{2\pi}} e^{-t^2/2} dt$. Unlike the exponential or Pareto, there is no closed-form expression for Φ^{-1} and numerical methods are required.

Newton's method. Given a target quantile u , we want to solve $\Phi(x) = u$ for x . Newton's method iterates:

$$x_{n+1} = x_n + (u - \Phi(x_n)) e^{x_n^2/2+c}, \quad c = \log \sqrt{2\pi},$$

since the derivative of Φ is the normal density $\phi(x_n) = e^{-x_n^2/2}/\sqrt{2\pi}$, so $1/\phi(x_n) = e^{x_n^2/2}\sqrt{2\pi}$. A good starting point is:

$$x_0 = \pm \sqrt{-1.6 \log(1.0004 - (1 - 2u)^2)},$$

and one Newton step achieves errors below 10^{-15} .

The **Beasley-Springer-Moro** (BSM) algorithm (Algorithm 22) is a highly optimised rational-function approximation used in practice. For central values $|u - 0.5| < 0.42$, it applies a rational function of $(u - 0.5)^2$. In the tails ($|u - 0.5| \geq 0.42$), it switches to a polynomial in $\log(-\log r)$ where

3.7.2 ITM for Normal Variables — Numerical Inversion of Φ II

$r = \min(u, 1 - u)$. This achieves absolute errors below 4×10^{-9} across the entire range $(0, 1)$ and is the basis of `scipy.stats.norm.ppf`.

Key Insight: When to Use ITM for Normals

The ITM approach via BSM or Newton is the method of choice when generating *large batches* of normal samples in a vectorized (SIMD) setting, because polynomial and rational approximations are highly parallelizable over arrays. Box-Muller and Marsaglia-Bray produce two samples per call and are better suited for sequential loops. NumPy's `rng.standard_normal()` uses a variant of this numerical inversion.

3.7.3 Multivariate Normal $\mathcal{N}(\boldsymbol{\mu}, \boldsymbol{\Sigma})$ I

A random vector $\mathbf{X} \in \mathbb{R}^d$ has the multivariate normal distribution $\mathcal{N}(\boldsymbol{\mu}, \boldsymbol{\Sigma})$ if every linear combination $\mathbf{a}^T \mathbf{X} = a_1 X_1 + \dots + a_d X_d$ is univariate normal.

Notation: Multivariate Normal

- $\boldsymbol{\mu}$ (mu): the d -dimensional *mean vector*, $\boldsymbol{\mu} = (\mu_1, \dots, \mu_d)^T$, where $\mu_i = \mathbb{E}[X_i]$.
- $\boldsymbol{\Sigma}$ (Sigma): the $d \times d$ *covariance matrix*. Its (i, j) entry is $\Sigma_{ij} = \text{Cov}(X_i, X_j) = \mathbb{E}[(X_i - \mu_i)(X_j - \mu_j)]$. The diagonal entries $\Sigma_{ii} = \text{Var}(X_i)$ are the individual variances.
- $\boldsymbol{\Sigma}$ must be symmetric and positive semidefinite (all eigenvalues ≥ 0).

The key structural fact is that any $\mathcal{N}(\boldsymbol{\mu}, \boldsymbol{\Sigma})$ vector can be obtained from a vector of independent standard normals by a linear transformation. This is the basis of the simulation algorithm.

3.7.3 Multivariate Normal $\mathcal{N}(\boldsymbol{\mu}, \boldsymbol{\Sigma})$ II

Proposition 3.7.8 — Cholesky Method for Multivariate Normal

Every symmetric positive semidefinite matrix $\boldsymbol{\Sigma}$ can be factored as $\boldsymbol{\Sigma} = \mathbf{A}\mathbf{A}^T$, where $\mathbf{A}$ is a lower-triangular matrix (the *Cholesky factor*). If $\mathbf{Z} = (Z_1, \dots, Z_d)^T$ is a vector with i.i.d. $Z_i \sim \mathcal{N}(0, 1)$, then:

$$\mathbf{X} = \mathbf{A}\mathbf{Z} + \boldsymbol{\mu} \sim \mathcal{N}(\boldsymbol{\mu}, \boldsymbol{\Sigma}).$$

Proof. $\mathbb{E}[\mathbf{X}] = \mathbf{A}\mathbb{E}[\mathbf{Z}] + \boldsymbol{\mu} = \boldsymbol{\mu}$ (since $\mathbb{E}[\mathbf{Z}] = 0$). $\text{Cov}(\mathbf{X}) = \mathbf{A} \text{Cov}(\mathbf{Z}) \mathbf{A}^T = \mathbf{A} \mathbf{I} \mathbf{A}^T = \mathbf{A}\mathbf{A}^T = \boldsymbol{\Sigma}$, where $\mathbf{I}$ is the identity matrix and $\text{Cov}(\mathbf{Z}) = \mathbf{I}$ because the components of $\mathbf{Z}$ are independent with unit variance.

How to Read $\mathbf{X} = \mathbf{A}\mathbf{Z} + \boldsymbol{\mu}$

The Cholesky factor $\mathbf{A}$ is a “stretching and rotating” matrix that transforms the spherically symmetric standard normal cloud $\mathbf{Z}$ into an ellipsoidal cloud aligned with the correlations specified by $\boldsymbol{\Sigma}$. Adding $\boldsymbol{\mu}$ shifts the centre of the cloud to the desired mean.

3.7.3 Multivariate Normal $\mathcal{N}(\boldsymbol{\mu}, \boldsymbol{\Sigma})$ III

Bivariate case ($d = 2$). With $\boldsymbol{\Sigma} = \begin{pmatrix} \sigma_1^2 & \rho\sigma_1\sigma_2 \\ \rho\sigma_1\sigma_2 & \sigma_2^2 \end{pmatrix}$, where ρ (rho) is the correlation coefficient ($-1 < \rho < 1$), σ_1 (sigma one) and σ_2 (sigma two) are the standard deviations of X_1 and X_2 respectively. Using the Cholesky factor, the simulation formula is:

$$X_1 = \sigma_1 \left(\sqrt{1 - |\rho|} Z_1 + \sqrt{|\rho|} Z_3 \right), \quad X_2 = \sigma_2 \left(\sqrt{1 - |\rho|} Z_2 + \text{sign}(\rho) \sqrt{|\rho|} Z_3 \right),$$

where Z_1, Z_2, Z_3 are three independent $\mathcal{N}(0, 1)$ random variables. The shared component Z_3 introduces the correlation between X_1 and X_2 .

3.7.3 Multivariate Normal $\mathcal{N}(\boldsymbol{\mu}, \boldsymbol{\Sigma})$ IV

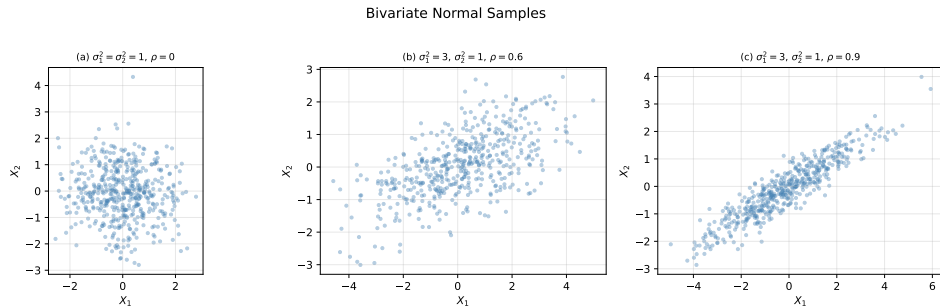


Figure: 500 samples from $\mathcal{N}(0, \boldsymbol{\Sigma})$ for three covariance settings. (a) $\sigma_1^2 = \sigma_2^2 = 1, \rho = 0$: the cloud is a symmetric circle (no correlation). (b) $\sigma_1^2 = 3, \sigma_2^2 = 1, \rho = 0.6$: the cloud is an elongated ellipse tilted toward the upper right. (c) $\rho = 0.9$: strong positive correlation, the cloud is very elongated.

3.7 Python: Multivariate Normal via Cholesky

Listing 4: Multivariate normal sampling via Cholesky decomposition (Proposition 3.7.8)

```
1 import numpy as np
2 rng = np.random.default_rng(42)
3
4 def sample_multivariate_normal(mu, Sigma, n, rng):
5     """
6     Generate n samples from N(mu, Sigma) using the Cholesky method.
7     Proposition 3.7.8:  $X = A*Z + \mu$ , where  $\Sigma = A @ A.T$ .
8     A is the lower-triangular Cholesky factor.
9     """
10    d = len(mu)
11    A = np.linalg.cholesky(Sigma)    # A is lower-triangular
12    Z = rng.standard_normal((d, n))  # d x n matrix of i.i.d. N(0,1)
13    return (A @ Z).T + mu            # shape: (n, d)
14
15 # 2D example: mean [1, -2], covariance with rho=0.849
16 mu = np.array([1.0, -2.0])
17 Sigma = np.array([[2.0, 1.2],
18                   [1.2, 1.0]])    # rho = 1.2/sqrt(2*1) = 0.849
19
20 samples = sample_multivariate_normal(mu, Sigma, 2000, rng)
21 print("Empirical mean:", samples.mean(axis=0).round(3))
22 # Expected: approx [1.0, -2.0]
23 print("Empirical covariance:\n", np.cov(samples.T).round(3))
24 # Expected: approx [[2.0, 1.2], [1.2, 1.0]]
```

Outline

- 1 3.1 Inverse Transform Method (ITM)
- 2 3.2 ITM Variants for Lattice Distributions
- 3 3.3 Ad Hoc Methods
- 4 3.4 Uniform Distribution on Geometric Shapes
- 5 3.5 Acceptance-Rejection Method
- 6 3.6 Composition Method
- 7 3.7 Generating Gaussian Random Numbers
- 8 3.8 Uniform Distribution on Balls, Spheres, Ellipsoids

3.8 Geometry: Balls, Spheres, and Their Volumes I

This section develops algorithms to sample uniformly from geometric objects — the d -ball, the $(d - 1)$ -sphere, and ellipsoids — without relying on the rejection-from-bounding-box method, which is exponentially inefficient in high dimensions (as we saw in Section 3.4).

Key Geometric Objects and Their Volumes

$$\begin{aligned} B_d &= \{x \in \mathbb{R}^d : x_1^2 + \cdots + x_d^2 \leq 1\} && \text{(the } d\text{-dimensional unit ball),} \\ S_{d-1} &= \{x \in \mathbb{R}^d : x_1^2 + \cdots + x_d^2 = 1\} && \text{(the } (d - 1)\text{-dimensional unit sphere).} \end{aligned}$$

Their volumes (measures) are:

$$\text{Vol}(B_d) = \frac{\pi^{d/2}}{\Gamma(d/2 + 1)}, \quad \text{Area}(S_{d-1}) = \frac{2\pi^{d/2}}{\Gamma(d/2)}.$$

Here Γ (Gamma) is the gamma function, satisfying $\Gamma(n) = (n - 1)!$ for positive integers n and $\Gamma(1/2) = \sqrt{\pi}$.

3.8 Geometry: Balls, Spheres, and Their Volumes II

Notation

- B_d (ball): all points *at or inside* the unit sphere.
- S_{d-1} (sphere): only points *on* the surface of the unit sphere. The sphere S_{d-1} is $(d-1)$ -dimensional (a surface), embedded in d -dimensional space $\mathbb{R}^d$. For example, S_2 is the familiar 2D surface of a ball in 3D space.
- $\pi^{d/2}$: pi to the power $d/2$; grows, but $\Gamma(d/2 + 1)$ grows faster, so $\text{Vol}(B_d) \rightarrow 0$ as $d \rightarrow \infty$.

Surface area element (Lemma 3.8.8). For a surface parametrised as $\mathbf{x}(t)$ with Jacobian matrix $J(t) = (\partial x_j / \partial t_i)_{i,j}$, the surface area element is $dA = \sqrt{|\det(JJ^T)|} dt$. This generalises the standard change-of-variables Jacobian to lower-dimensional surfaces embedded in $\mathbb{R}^d$.

3.8.1 Sampling Uniformly on S_2 and in B_3 I

For $d = 3$, the sphere S_2 (the surface of the unit ball in 3D) is parametrised by spherical coordinates: $(\theta, \phi) \in [0, 2\pi) \times [0, \pi)$, with

$$x = \cos \theta \sin \phi, \quad y = \sin \theta \sin \phi, \quad z = \cos \phi.$$

Here θ (theta) is the azimuthal angle (longitude) and ϕ (phi) is the polar angle (colatitude, measured from the north pole). The surface area element is $dA = \sin \phi \, d\phi \, d\theta$.

Important: sampling ϕ uniformly on $[0, \pi)$ would *not* give a uniform distribution on the sphere! Points near the poles ($\phi \approx 0$ or $\phi \approx \pi$) are closer together (the area element $\sin \phi$ is small there), so they need to be sampled less frequently. The correct density for ϕ is $f(\phi) = \frac{1}{2} \sin \phi$, which has c.d.f. $F_\phi(\phi) = (1 - \cos \phi)/2$. Inverting: $\phi = \arccos(1 - 2V)$ for $V \sim \mathcal{U}[0, 1)$. The azimuthal angle $\theta = 2\pi W$ for independent $W \sim \mathcal{U}[0, 1)$.

3.8.1 Sampling Uniformly on S_2 and in B_3 II

Proposition 3.8.6 — The Normal Trick for $\mathcal{U}(S_2)$

If $Z_1, Z_2, Z_3 \stackrel{\text{i.i.d.}}{\sim} \mathcal{N}(0, 1)$, then the normalised vector

$$\mathbf{x} = \frac{(Z_1, Z_2, Z_3)}{\|(Z_1, Z_2, Z_3)\|} = \left(\frac{Z_1}{\|Z\|}, \frac{Z_2}{\|Z\|}, \frac{Z_3}{\|Z\|} \right) \sim \mathcal{U}(S_2).$$

Why this works. The density of $Z = (Z_1, Z_2, Z_3)$ is $f(z) \propto e^{-\|z\|^2/2}$, which depends only on the radius $\|z\|$ and not on the direction $z/\|z\|$. This *spherical symmetry* means that the direction is uniformly distributed on S_2 , independent of the radius. Normalising removes the radius, leaving a $\mathcal{U}(S_2)$ sample.

3.8.1 Sampling Uniformly on S_2 and in B_3 III

The Normal trick generalises immediately to any dimension d .

General Algorithms: $\mathcal{U}(S_{d-1})$ and $\mathcal{U}(B_d)$

Sampling uniformly on the $(d-1)$ -sphere S_{d-1} :

- 1 Generate $Z = (Z_1, \dots, Z_d)^T$ with i.i.d. $Z_i \sim \mathcal{N}(0, 1)$.
- 2 Return $X = Z/\|Z\|$.

Sampling uniformly in the d -ball B_d (Proposition 3.8.13):

- 1 Sample $S \sim \mathcal{U}(S_{d-1})$ using the steps above.
- 2 Independently sample $U \sim \mathcal{U}[0, 1]$.
- 3 Return $X = U^{1/d} S$.

The scaling $U^{1/d}$ (U to the power one over d) ensures uniformity inside the ball. The c.d.f. of the radius $R = \|X\|$ must be $F_R(r) = r^d$ for $r \in [0, 1]$ (since volume scales as r^d), and ITM gives $R = U^{1/d}$.

3.8.1 Sampling Uniformly on S_2 and in B_3 IV

How to Read the Scaling $U^{1/d}$

In one dimension ($d = 1$), $U^{1/1} = U$ — a uniform radius, which is correct for a line segment. In two dimensions ($d = 2$), $U^{1/2} = \sqrt{U}$ — most of the area of a disc is near the boundary, so the radius must be biased toward 1. In high dimensions, $U^{1/d}$ concentrates near 1 for large d , reflecting the fact that almost all of the volume of a high-dimensional ball is near the surface (the “hollow ball” phenomenon).

3.8.1 Sampling Uniformly on S_2 and in B_3 V

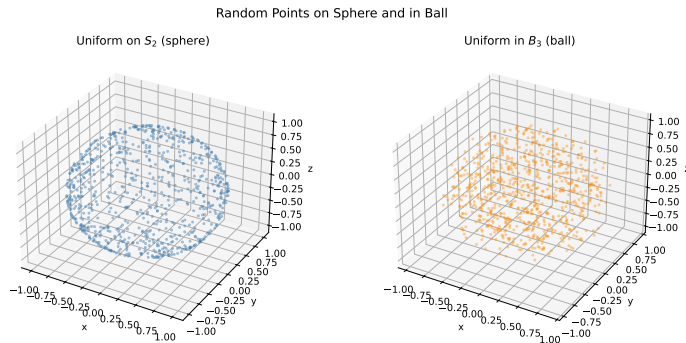


Figure: Left: 1000 uniform random points on S_2 (the unit sphere surface in 3D) generated by the Normal trick. Right: 1000 uniform random points inside B_3 (the unit ball in 3D) generated by the Normal trick followed by scaling by $U^{1/3}$.

3.8 Python: Sphere and Ball Sampling

Listing 5: Uniform sampling on the $(d-1)$ -sphere and d -ball via the Normal trick

```
1 import numpy as np
2 rng = np.random.default_rng(42)
3
4 def sample_sphere(d, n, rng):
5     """
6     Uniform on the  $(d-1)$ -sphere  $S_{\{d-1\}}$  in  $\mathbb{R}^d$ .
7     Method: normalise  $d$  independent  $N(0,1)$  variables.
8     The spherical symmetry of  $N(0,1)$  ensures uniform direction.
9     """
10    Z = rng.standard_normal((n, d))
11    norms = np.linalg.norm(Z, axis=1, keepdims=True)
12    return Z / norms      # each row has norm exactly 1
13
14 def sample_ball(d, n, rng):
15     """
16     Uniform inside the  $d$ -ball  $B_d$  in  $\mathbb{R}^d$ .
17     Method:  $X = U^{(1/d)} * (\text{point on sphere})$ .
18     Scaling  $U^{(1/d)}$  gives CDF  $F_R(r) = r^d$  on  $[0,1]$ ,
19     matching the volume growth  $r^d$  of a  $d$ -ball.
20     """
21    S = sample_sphere(d, n, rng)      # uniform direction
22    U = rng.uniform(0, 1, n)[:, None] # uniform radius before scaling
23    return S * (U ** (1.0 / d))        # scale to get uniform in ball
24
25 # Verification
```


3.8.3 Sampling Uniformly Within an Ellipsoid I

An ellipsoid $\mathcal{E}_d = \{x \in \mathbb{R}^d : x^T A^{-1} x \leq 1\}$ is a linearly stretched version of the unit ball, where A is a symmetric positive-definite matrix. The volume of $\mathcal{E}_d$ is $\text{Vol}(B_d) \cdot \sqrt{\det A}$.

Notation: $x^T A^{-1} x$ (x -transpose times A -inverse times x) is a *quadratic form*; it measures the “distance” of x from the origin in the metric defined by A^{-1} . The condition $x^T A^{-1} x \leq 1$ defines the interior of the ellipsoid.

Key idea. Factorise $A = LL^T$ (Cholesky decomposition). The map $x \mapsto L^{-1}x$ transforms $\mathcal{E}_d$ bijectively (one-to-one and onto) into the unit ball B_d . Therefore: sample $X \sim \mathcal{U}(B_d)$ and return $Y = LX$, which is then uniform in $\mathcal{E}_d$.

Algorithm 23 — $\mathcal{U}(\mathcal{E}_d)$: Uniform in an Ellipsoid

- 1 Factorise $A = LL^T$ (Cholesky decomposition, or spectral $A = P \text{diag}(a^2) P^T$).
- 2 Generate $X \sim \mathcal{U}(B_d)$ using the Normal trick.
- 3 Return $Y = LX$.

3.8.3 Sampling Uniformly Within an Ellipsoid II

Worked 2D Example

For the ellipse $(x/3)^2 + (y/1)^2 \leq 1$ with semi-axes $a = 3$ and $b = 1$: the matrix is $A = \text{diag}(9, 1)$ and $L = \text{diag}(3, 1)$. Sample $(X_1, X_2) \sim \mathcal{U}(B_2)$, then return $(Y_1, Y_2) = (3X_1, X_2)$. This stretches the circular sample by a factor of 3 in the x -direction.

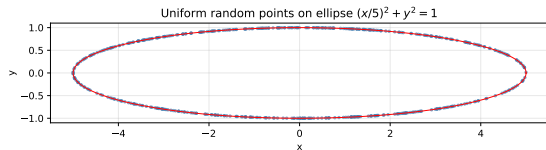


Figure: 1 000 uniform random points within the ellipse $(x/3)^2 + y^2 \leq 1$ generated by Algorithm 23: sample uniformly in the unit disc, then apply the linear map $(X_1, X_2) \mapsto (3X_1, X_2)$.

3.8.4–3.8.5 Sampling on Ellipsoid Surfaces and Average Shadow I

Sampling on the surface of a 3D ellipsoid. Consider the ellipsoid surface $\frac{x^2}{a^2} + \frac{y^2}{b^2} + \frac{z^2}{c^2} = 1$, parametrised by $(\theta, \phi) \in [0, 2\pi) \times [0, \pi)$:

$$\mathbf{x}(\theta, \phi) = (a \cos \theta \sin \phi, b \sin \theta \sin \phi, c \cos \phi).$$

The area element $dA = \sqrt{g(\phi, \theta)} d\phi d\theta$ where

$\sqrt{g} = \sin \phi \sqrt{c^2(a^2 \sin^2 \phi \sin^2 \theta + b^2 \sin^2 \phi \cos^2 \theta) + a^2 b^2 \cos^2 \phi}$ does *not* separate into a product of a function of θ and a function of ϕ when a, b, c are all different. Therefore, θ and ϕ cannot be sampled independently. Algorithm 24 uses a numerical approach: subdivide the parameter rectangle into a grid, approximate each cell's area using a cross-product of tangent vectors, then sample a cell proportional to its area and place a uniform point within it.

Average shadow of a 3D body (Section 3.8.5). The *shadow* (projection) of a planar face F cast by parallel light arriving from direction $\mathbf{n}$ has area equal to $|\cos \angle(\mathbf{n}, \hat{\mathbf{n}}_F)| \cdot \text{Area}(F)$, where $\hat{\mathbf{n}}_F$ is the outward normal of the face.

3.8.4–3.8.5 Sampling on Ellipsoid Surfaces and Average Shadow II

Expected Shadow Formula

For a single flat face F with the light direction drawn uniformly from S_2 :

$$\mathbb{E}[\text{Area}(\text{shadow}(F))] = \frac{\text{Area}(F)}{2}.$$

For a convex polyhedron with faces $F_1, \dots, F_m$ and light direction drawn uniformly from all directions:

$$\mathbb{E}[\text{Area}(\text{shadow})] = \frac{\text{surface area}}{4}.$$

For a unit cube (surface area = 6): $\mathbb{E}[\text{shadow}] = 6/4 = 3/2$.

Derivation. For a single face, we need $\mathbb{E}[|\cos \phi| \sin \phi]$ where the polar angle ϕ (phi) has density $f(\phi) = \frac{1}{2} \sin \phi$ on $[0, \pi)$ (uniform on S_2). This integral evaluates to $1/2$, giving the factor of $1/2$ for each face. Summing over all faces and noting that each face points “outward” for half of all light directions (the other half hit the opposite side and cast no shadow) gives the overall factor of $1/4$.

Outline

- 1 3.1 Inverse Transform Method (ITM)
- 2 3.2 ITM Variants for Lattice Distributions
- 3 3.3 Ad Hoc Methods
- 4 3.4 Uniform Distribution on Geometric Shapes
- 5 3.5 Acceptance-Rejection Method
- 6 3.6 Composition Method
- 7 3.7 Generating Gaussian Random Numbers
- 8 3.8 Uniform Distribution on Balls, Spheres, Ellipsoids

3.9.1 Conditional Inverse Transform Method I

When we need to simulate a *bivariate* random vector (X, Y) with a specified joint density $f(x, y)$, one natural approach is the *conditional ITM*: first simulate the marginal distribution of X , then simulate Y from the conditional distribution of Y given X . This leverages the fundamental identity $f(x, y) = f_X(x) \cdot f_{Y|X=x}(y)$ to decompose the joint sampling into two simpler one-dimensional problems.

Algorithm — Conditional ITM for (X, Y)

- 1 Compute the marginal density $f_X(x) = \int_{-\infty}^{+\infty} f(x, y) dy$ and its c.d.f. F_X .
- 2 Generate $X = F_X^{\leftarrow}(U_1)$ for independent $U_1 \sim \mathcal{U}[0, 1)$.
- 3 Given $X = x$, compute the conditional c.d.f. $F_{Y|X=x}(y) = \int_{-\infty}^y \frac{f(x, t)}{f_X(x)} dt$.
- 4 Generate $Y = F_{Y|X=x}^{\leftarrow}(U_2)$ for independent $U_2 \sim \mathcal{U}[0, 1)$.
- 5 Return (X, Y) .

3.9.1 Conditional Inverse Transform Method II

Example 3.9.1 — Worked Derivation

Let $f(x, y) = x e^{-x(y+1)}$ for $x, y \geq 0$.

Step 1: find the marginal f_X .

$$f_X(x) = \int_0^{\infty} x e^{-x(y+1)} dy = x e^{-x} \int_0^{\infty} e^{-xy} dy = x e^{-x} \cdot \frac{1}{x} = e^{-x}.$$

So $X \sim \text{Exp}(1)$. Simulation: $X = -\log U_1$.

Step 2: find the conditional $f_{Y|X=x}$.

$$f_{Y|X=x}(y) = \frac{f(x, y)}{f_X(x)} = \frac{x e^{-x(y+1)}}{e^{-x}} = x e^{-xy}.$$

This is the density of $\text{Exp}(x)$. Simulation: $Y = -\log(U_2)/x$.

3.9.1 Conditional Inverse Transform Method III

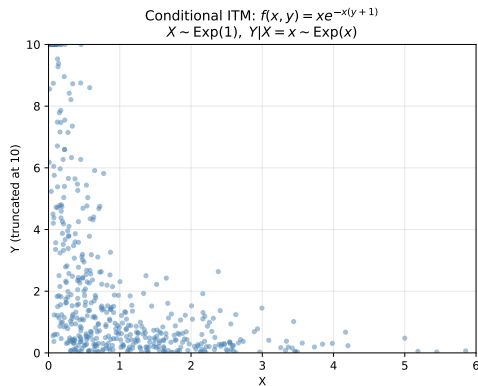


Figure: 100 samples from $f(x, y) = xe^{-x(y+1)}$ via the conditional ITM. Large values of X lead to $\text{Exp}(X)$ distributions for Y that concentrate near 0, explaining the positive correlation visible in the scatter plot.

3.9.2 Bivariate Copulas and Sklar's Theorem I

Specifying a joint distribution via its density intertwines the marginal distributions (how each variable individually behaves) and the dependence structure (how the variables co-vary). *Copulas* provide a language for separating these two concerns: a copula encodes only the dependence structure, and the marginal distributions are specified separately.

Definition — Bivariate Copula

A **copula** is a bivariate c.d.f. $C : [0, 1]^2 \rightarrow [0, 1]$ whose marginal distributions are both $\mathcal{U}[0, 1]$. Formally, C satisfies:

- a Boundary conditions: $C(u, 0) = C(0, v) = 0$ and $C(u, 1) = u$, $C(1, v) = v$.
- b 2-increasing: for $u_1 \leq u_2$, $v_1 \leq v_2$: $C(u_2, v_2) - C(u_1, v_2) - C(u_2, v_1) + C(u_1, v_1) \geq 0$. (This ensures C is a valid c.d.f., assigning non-negative probability to every rectangle.)

3.9.2 Bivariate Copulas and Sklar's Theorem II

Theorem 3.9.2 — Sklar's Theorem

For any joint c.d.f. $F_{X,Y}(t_1, t_2)$ with marginals F_X and F_Y , there exists a copula C such that:

$$F_{X,Y}(t_1, t_2) = C(F_X(t_1), F_Y(t_2)).$$

If F_X and F_Y are continuous, C is *unique*. The joint density factors as:

$$f_{X,Y}(t_1, t_2) = f_X(t_1) \cdot f_Y(t_2) \cdot c(F_X(t_1), F_Y(t_2)),$$

where $c(u_1, u_2) = \partial^2 C(u_1, u_2) / \partial u_1 \partial u_2$ is the **copula density**.

How to Read Sklar's Theorem

Sklar's theorem says any joint distribution can be built in two steps: (1) choose the marginal distributions F_X and F_Y independently, (2) choose a copula C to specify the dependence structure. The result is always a valid joint distribution. Crucially, you can change the copula without changing the marginals, or change the marginals without changing the dependence structure.

3.9.2 Bivariate Copulas and Sklar's Theorem III

Fréchet-Hoeffding Bounds (Theorem 3.9.6)

For any bivariate copula $C(u_1, u_2)$:

$$\max(u_1 + u_2 - 1, 0) \leq C(u_1, u_2) \leq \min(u_1, u_2).$$

The three extreme copulas are:

- **Upper bound (perfect positive dependence):** $C_U(u_1, u_2) = \min(u_1, u_2)$. Simulation: set $V_1 = V_2 = U$ for a single $U \sim \mathcal{U}[0, 1]$. The two marginals move in lockstep (perfectly correlated).
- **Lower bound (perfect negative dependence):** $C_L(u_1, u_2) = \max(0, u_1 + u_2 - 1)$. Simulation: set $V_1 = U$, $V_2 = 1 - U$. When one is large, the other is small.
- **Independence:** $C_I(u_1, u_2) = u_1 u_2$. Simulation: set V_1, V_2 independent.

3.9.2 Bivariate Copulas and Sklar's Theorem IV

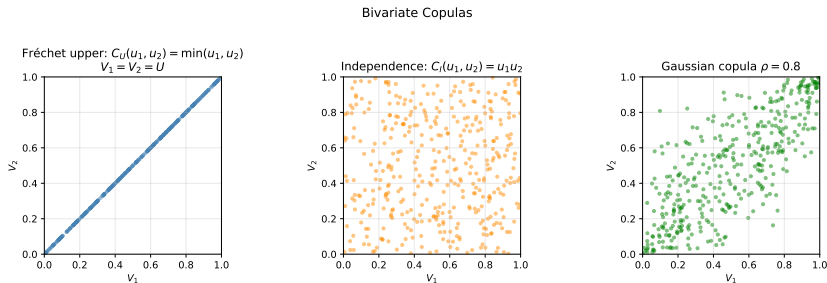


Figure: 1 000 samples (V_1, V_2) from three copulas. Left: the Fréchet upper copula C_U (all points on the diagonal $V_1 = V_2$, perfect positive dependence). Centre: the independence copula C_I (uniform scatter). Right: the Gaussian copula with $\rho = 0.8$ (elliptical cloud, positive dependence but not perfect).

3.9.3 Classical Copulas and Their Simulation I

Gaussian (Normal) Copula

Let Φ_ρ (Phi sub rho) be the bivariate normal c.d.f. with standard marginals and correlation ρ (rho) $\in (-1, 1)$. The Gaussian copula is:

$$C_\rho(u_1, u_2) = \Phi_\rho(\Phi^{-1}(u_1), \Phi^{-1}(u_2)).$$

Here Φ^{-1} (capital Phi inverse) is the standard normal quantile function.

Simulation of $(V_1, V_2) \sim C_\rho$:

- 1 Generate $(X, Y) \sim \mathcal{N}(0, \Sigma)$ with $\Sigma = \begin{pmatrix} 1 & \rho \\ \rho & 1 \end{pmatrix}$ using the Cholesky method.
- 2 Transform to uniform marginals: $(V_1, V_2) = (\Phi(X), \Phi(Y))$.

3.9.3 Classical Copulas and Their Simulation II

Notation for the Gaussian Copula

- $\Phi^{-1}(u)$: the u -quantile of the standard normal; converts a uniform probability $u \in (0, 1)$ into a normal value.
- $\Phi(x)$: the normal c.d.f.; converts a normal value back to a probability in $(0, 1)$.
- The round-trip $u \mapsto \Phi^{-1}(u) \mapsto \text{bivariate normal} \mapsto \Phi(\cdot)$ maps the copula sample to uniform marginals.
- As $\rho \rightarrow 0$: $C_\rho \rightarrow C_I$ (independence). As $\rho \rightarrow 1$: $C_\rho \rightarrow C_U$ (perfect positive dependence). As $\rho \rightarrow -1$: $C_\rho \rightarrow C_L$ (perfect negative dependence).

Transforming copula samples to arbitrary marginals. Given (V_1, V_2) sampled from any copula C , we obtain samples with arbitrary marginals F_X and F_Y via the ITM: $X' = F_X^{\leftarrow}(V_1)$ and $Y' = F_Y^{\leftarrow}(V_2)$. By Sklar's theorem, (X', Y') has joint distribution $C(F_X, F_Y)$ with the specified marginals.

3.9.3 Classical Copulas and Their Simulation III

Example 3.9.10 — Mixing Exponential and Pareto Marginals

We want (X', Y') with $X' \sim \text{Exp}(1)$, $Y' \sim \text{Par}(1)$, and the Gaussian copula with $\rho = 0.8$.

- ❶ Sample (V_1, V_2) from the Gaussian copula $C_{0.8}$.
- ❷ Apply ITM: $X' = F_{\text{Exp}}^{\leftarrow}(V_1) = -\log(1 - V_1)$.
- ❸ Apply ITM: $Y' = F_{\text{Par}}^{\leftarrow}(V_2) = \frac{1}{1-V_2} - 1$.

The result has the heavy-tailed Pareto and light-tailed exponential marginals, but with 0.8-correlated dependence structure.

3.9.3 Classical Copulas and Their Simulation IV

Gaussian Copula Samples

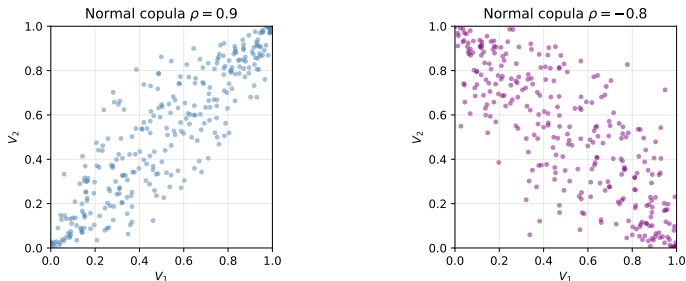


Figure: Gaussian copula samples: $\rho = 0.9$ (left, strong positive dependence, most points near the $V_1 = V_2$ diagonal) vs. $\rho = -0.8$ (right, strong negative dependence, most points near the anti-diagonal $V_1 + V_2 \approx 1$).

3.9.3 Classical Copulas and Their Simulation V

Key Insight: Why Copulas Are Useful

Copulas provide a *modular* framework for multivariate simulation:

- **Marginals** are fitted to univariate data (e.g. fit an exponential to claim arrival times, a Pareto to claim sizes).
- **Dependence** is modelled by a copula fitted to the bivariate rank correlation of the data.
- Sklar's theorem guarantees the combination is always a valid joint distribution.

This modular approach is widely used in finance (credit risk modelling of multiple defaults), insurance (modelling correlated aggregate losses), and environmental science (modelling joint extremes such as simultaneous high temperature and drought). The key advantage is that changing the copula does not alter the marginal distributions, and vice versa.

3.9 Python: Gaussian Copula

Listing 6: Gaussian copula simulation with arbitrary marginals (Example 3.9.10)

```
1 import numpy as np
2 from scipy.stats import norm
3
4 rng = np.random.default_rng(42)
5
6 def sample_gaussian_copula(rho, n, rng):
7     """
8     Sample (V1, V2) from the Gaussian copula with correlation rho.
9     Step 1: Generate bivariate normal with corr. rho via Cholesky.
10    Step 2: Apply normal CDF to each component to get U[0,1] marginals.
11    """
12    Sigma = np.array([[1.0, rho], [rho, 1.0]])
13    A = np.linalg.cholesky(Sigma) # Sigma = A @ A.T
14    Z = rng.standard_normal((2, n))
15    W = A @ Z # bivariate normal with corr rho
16    V1 = norm.cdf(W[0]) # transform to uniform marginals
17    V2 = norm.cdf(W[1])
18    return V1, V2
19
20 def transform_marginals(V1, V2):
21     """Transform copula samples (V1, V2) to Exp(1) x Par(1) marginals."""
22    X = -np.log(1 - V1) # F_Exp^{-1}(V1): Exp(1) quantile
23    Y = 1.0 / (1 - V2) - 1 # F_Par^{-1}(V2): Par(alpha=1) quantile
24    return X, Y
25
```

Outline

- 1 3.1 Inverse Transform Method (ITM)
- 2 3.2 ITM Variants for Lattice Distributions
- 3 3.3 Ad Hoc Methods
- 4 3.4 Uniform Distribution on Geometric Shapes
- 5 3.5 Acceptance-Rejection Method
- 6 3.6 Composition Method
- 7 3.7 Generating Gaussian Random Numbers
- 8 3.8 Uniform Distribution on Balls, Spheres, Ellipsoids

3.10 Can We See the “Heaviness” of a Distribution? I

The tail behaviour of a distribution has dramatic practical consequences for Monte Carlo estimation. We now illustrate this through simulation, comparing running averages of sequences drawn from several distributions that all have the *same* theoretical mean $\mathbb{E}[X_i] = 1$ (where the mean is finite), but with very different tail behaviours.

Distributions Compared (all with mean = 1)

Distribution	Tail	Variance
$\mathcal{N}(1, 1)$	Light	1
$\text{Exp}(1)$	Light	1
$\text{LN}(-1/2, 1)$	Moderate	≈ 4.67
$\text{W}(1/2, \sqrt{2})$	Moderate	Finite
$\text{Par}(1.2)$ scaled	Very heavy	∞

Here $\text{LN}(\mu, \sigma)$ is the log-normal with log-parameters μ (mu) and σ (sigma), and $\text{W}(k, s)$ is the Weibull with shape k and scale s .

What to Expect

For light-tailed distributions ($\mathcal{N}$, Exp): the running average $S_n/n = (X_1 + \dots + X_n)/n$ converges smoothly toward the true mean of 1, with fluctuations shrinking like $1/\sqrt{n}$ (the CLT rate).

For $\text{Par}(\alpha)$ with $\alpha \leq 2$: the variance is *infinite*, so the CLT convergence rate $1/\sqrt{n}$ does not apply. Running averages converge far more slowly and show sudden large jumps whenever a very large value X_i is drawn.

3.10 Can We See the “Heaviness” of a Distribution? II

Light vs Heavy Tails

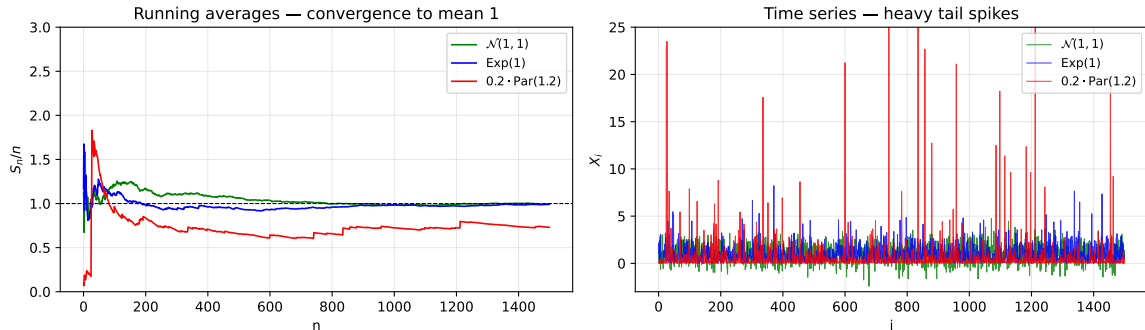


Figure: (a) Running averages S_n/n for $n = 1, \dots, 5000$ samples. The Normal (blue) and Exp (orange) running averages converge quickly to 1. The Pareto(1.2) (red) shows persistent large fluctuations and sudden upward

3.10 Can We See the “Heaviness” of a Distribution? III

jumps whenever a very large value is drawn. (b) The corresponding time series of individual X_i values. The Pareto draws include rare but enormous spikes that dominate the running sum, preventing rapid convergence.

3.10 Can We See the “Heaviness” of a Distribution? IV

QQ Plots for Tail Detection

A *quantile-quantile* (QQ) plot compares the *sample quantiles* of observed data against the *theoretical quantiles* of a reference distribution (usually $\mathcal{N}(0, 1)$ after standardisation). The procedure: sort the data $X_{(1)} \leq X_{(2)} \leq \dots \leq X_{(n)}$; the k -th point is $(X_{(k)}, \Phi^{-1}(k/(n+1)))$.

- **Light-tailed data:** the QQ plot points lie approximately on a straight line through the origin. This means the sample has similar extreme values to what a normal distribution would predict.
- **Heavy-tailed data:** the upper-right corner curves steeply upward (the sample has much larger maxima than the normal predicts). Similarly, the lower-left corner may curve downward for heavy left tails.
- **Skewed data:** only one tail curves away from the line.

3.10 Can We See the “Heaviness” of a Distribution? V

QQ Plots: Detecting Tail Behaviour

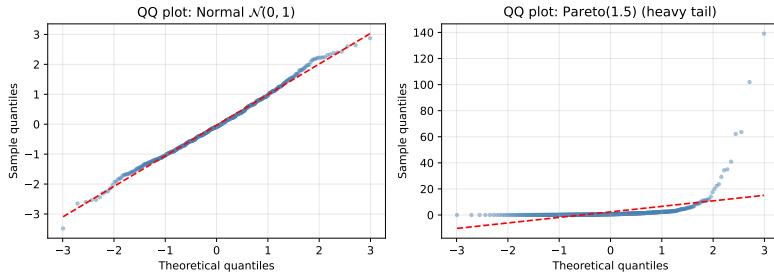


Figure: QQ plots against the $\mathcal{N}(0, 1)$ reference. Left: $\mathcal{N}(0, 1)$ data — the points lie on a straight line, confirming normality. Right: $\text{Par}(1.5)$ data — the upper-right corner curves sharply upward, indicating the distribution has a much heavier right tail than normal.

3.10 Can We See the “Heaviness” of a Distribution? VI

Key Takeaway: Tail Heaviness Is Observable

Heavy-tailedness is not just a theoretical property — it is directly visible in simulations. QQ plots reveal it in the shape of the scatter, and running averages reveal it through slow convergence and sudden large jumps. In Monte Carlo estimation with heavy-tailed estimators (i.e. estimators with heavy-tailed distributions), no practical sample size may be large enough to give reliable results without variance reduction techniques (Chapter 5).

Outline

- 1 3.1 Inverse Transform Method (ITM)
- 2 3.2 ITM Variants for Lattice Distributions
- 3 3.3 Ad Hoc Methods
- 4 3.4 Uniform Distribution on Geometric Shapes
- 5 3.5 Acceptance-Rejection Method
- 6 3.6 Composition Method
- 7 3.7 Generating Gaussian Random Numbers
- 8 3.8 Uniform Distribution on Balls, Spheres, Ellipsoids

Chapter 3 — Summary of All Methods I

This chapter has developed a complete toolkit for generating random variables from arbitrary distributions, using only uniform random numbers as a primitive. The table below gives a concise reference for all methods covered.

Chapter 3 — Summary of All Methods II

Complete Method Reference Table

Method	Algorithm	Key idea
ITM (general)	6	$X = F^{\leftarrow}(U)$; works when $F^{\leftarrow}$ is computable
ITM-Exp	7	$X = -\log(U)/\lambda$
ITM-Par	8	$X = U^{-1/\alpha} - 1$
Geometric closed form	–	$X = \lfloor \log U / \log p \rfloor$
ITM-d (discrete)	9	Sequential accumulation; $O(\mathbb{E}Y)$ cost
ITR (ratio recursion)	10	Ratio $c_{k+1} = p_{k+1}/p_k$; $(a, b, 0)$ class
DB (direct binomial)	11	Sum n Bernoulli trials; $O(n)$ cost
Erlang sum	–	$X = -\log(\prod U_i)/\lambda$; stable via log-sum
DP (Poisson via renewal)	12	Count Exp(1) inter-arrivals; $O(\lambda)$ cost
AR-d	14	$p_j \leq cq_j$; accept with probability

3.11 Bibliographical Notes

- **Devroye (1986)** — *Non-Uniform Random Variate Generation*. The definitive reference for all methods in this chapter, with detailed proofs and efficiency analyses.
- **Knuth (1997)** — *The Art of Computer Programming, Vol. 2*. Classical treatment of the inverse transform and discrete methods.
- **Glasserman (2004)** — *Monte Carlo Methods in Financial Engineering*. Covers ITM for normal variables and the Beasley-Springer-Moro algorithm.
- **Asmussen & Glynn (2007)** — *Simulation Algorithms for Stochastic Systems*. Fréchet-Hoeffding bounds and copulas.
- **Box & Muller (1958)** — Original paper introducing Algorithm 20.
- **Marsaglia & Bray (1964)** — Original paper introducing Algorithm 21.
- **Madras (2002)** — AR-c method for Gamma distributions.
- **Wichura (1988), AS241** — Algorithm underlying `scipy.stats.norm.ppf`.
- **Embrechts & Hofert** — Properties of copulas and generalized inverses.
- **Dagpunar (2007)** — Composition method, atomic physics example.
- **Ross (2006)** — Elementary introduction to simulation methods.